

Investigating the Numerical Stability of Polynomial Basis Representations in Constrained Probabilistic Inference

Kanishka Roy



Master of Science
School of Informatics
University of Edinburgh
2026

Abstract

The Probabilistic Algebraic Layer (PAL) performs constrained probabilistic inference by integrating polynomial probability densities over regions defined by algebraic constraints [1]. Its current numerical path uses monomial representations, whose conditioning can deteriorate with degree, dimension, and coordinate scale [2, 3, 4]. This dissertation evaluates Legendre and Chebyshev representations as numerically robust alternatives while holding the underlying mathematical polynomial fixed. Experiments cover analytical simplex integration, increasing degree and dimension, physical scaling, reduced precision, coefficient perturbations and recovery, constrained polytopes, partition functions, normalised queries, and a representative PAL pipeline. Orthogonal representations frequently improve conditioning, numerical-rank retention, and resistance to coefficient noise, but these advantages do not guarantee smaller forward error: unstable coefficient conversion or recovery can dominate the complete computation. A PAL-originated precision stress case additionally shows that reduced precision can reverse the sign of a mathematically non-negative spline-cell integral. The results therefore identify numerical stability as a pipeline property requiring stable representation, coefficient construction, constrained integration, and normalisation.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects requiring approval from the Informatics Research Ethics Committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(Kanishka Roy)

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Antonio Vergari, and my co-supervisor, Leander Kurscheidt, for their guidance, advice, and encouragement throughout this dissertation. Their expertise and feedback were invaluable in shaping both the direction of this research and the development of this work. I would also like to thank the University of Edinburgh and the School of Informatics for providing an outstanding academic environment and the resources that made this research possible. I am especially grateful for the opportunity to undertake this project within the MSc Informatics programme. Finally, I would like to thank my parents for their unwavering support, encouragement, and patience throughout my studies.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Question	2
1.3	Experimental Approach and Contributions	2
1.4	Scope and Structure	3
2	Background	4
2.1	Constrained Probabilistic Inference and PAL	4
2.1.1	Running Constraint Example	5
2.2	Polynomial Representations and Scaling	5
2.2.1	Monomial, Legendre, and Chebyshev Bases	5
2.2.2	Canonical Coordinate Scaling	6
2.3	Conditioning Diagnostics	6
2.4	Analytical Simplex Reference	7
2.5	Perturbation, Error, and Positive-Density Metrics	8
2.6	Representation and Pipeline Stability	8
3	Methodology	10
3.1	Research Questions and Common Design	10
3.1.1	Equivalent Polynomial Construction	11
3.1.2	Coefficient Scaling	11
3.2	Conditioning, Scale, Dimension, and Recovery	11
3.2.1	Sampled Conditioning Benchmarks	11
3.2.2	Constraint-Induced Conditioning	12
3.2.3	Coefficient Recovery	13
3.3	Precision, Perturbation, and Simplex Integration	13
3.4	Downstream Probability Experiment	13

3.5	Constrained-Polytope Experiments	14
3.5.1	Static and Changing Constraints	14
3.5.2	Noisy Coefficient Recovery	15
3.6	Representative PAL Evaluation	15
3.6.1	Supplementary PAL Precision Stress Case	15
3.7	Controls, Runtime, and Interpretation	16
4	Implementation	17
4.1	Overview	17
4.2	Basis Representation and Conversion	17
4.3	Conditioning and Constraint-Induced Geometry	18
4.4	Analytical Integration	19
4.5	Sampling, Perturbation, and Density Construction	19
4.6	Downstream Probability Computation	20
4.7	Constrained Domains and Coefficient Recovery	20
4.8	Representative PAL Path	21
4.9	Validation and Reproducibility	21
5	Experimental Evaluation	23
5.1	Experimental Setup and Validation	23
5.2	Conditioning with Degree and Constraint Geometry	24
5.3	Physical Scale and Dimension	26
5.4	Reduced Precision	27
5.5	Downstream Probability Evaluation	28
5.6	Constrained Polytope and Coefficient Recovery	29
5.7	Representative PAL Evaluation	31
5.7.1	Conditioning and Downstream Error	31
5.7.2	Precision-Sensitive PAL Triangle Case Study	32
5.8	Experimental Synthesis	33
5.9	Limitations	34
6	Discussion	36
6.1	Overview	36
6.2	RQ1: How Does Basis Choice Affect Numerical Conditioning?	36
6.3	RQ2: When Does Conditioning Become Practically Important?	37
6.4	RQ3: Do Orthogonal Bases Improve Constrained Integration?	38

6.5	RQ4: Do Numerical Effects Reach Probabilistic Inference?	38
6.6	Conditioning Is Necessary but Not Sufficient	39
6.7	Implications for PAL	40
6.8	Relation to Prior Work and Future Directions	40
6.9	Summary	41
7	Conclusion	42
7.1	Summary and Answer to the Research Question	42
7.2	Implications and Future Work	43
7.3	Final Conclusion	43
	Bibliography	45
A	Validation and Verification	47
A.1	Representative Smoke Tests	48
A.2	Constraint-Induced Gram Validation	49
B	Generated Polynomial Examples	50
B.1	Two-Dimensional Surface Views	50
B.2	Cross-Basis Equivalence	50
B.3	Representative Constrained Region	51
B.4	Supplementary Benchmark Figures	51
C	Selected Numerical Tables	61
C.1	Degree Benchmark	61
C.2	Physical-Scale Benchmark	62
C.3	Higher-Dimensional Benchmark	62
C.4	Reduced Precision	63
C.5	Downstream Probability Benchmark	63
C.6	Constraint-Induced Conditioning	64
D	Selected Implementation Excerpts	65
D.1	Analytical Simplex Moment	65
D.2	Condition Number	65
D.3	Coefficient Perturbation	66
D.4	Basis Count	67

E	Reproducibility	68
E.1	Benchmark Commands	68
E.2	Constraint-Induced Conditioning	68
E.3	Plotting and Generated-Polynomial Commands	69
E.4	Result Artefacts	70
E.5	Repository	71
E.6	Final Software Environment	71
E.7	Reproducibility Record	71

Chapter 1

Introduction

1.1 Motivation

Machine-learning systems may be required to satisfy physical, logical, safety, or geometric constraints that are not captured completely by training data [5, 6, 7]. The Probabilistic Algebraic Layer (PAL) incorporates algebraic constraints into probabilistic inference using learnable piecewise-polynomial densities and linear-real-arithmetic constraints [1]. For an unnormalised density $p(\mathbf{x})$ and feasible region $C \subseteq \mathbb{R}^n$, PAL requires the constrained mass

$$Z_C = \int_C p(\mathbf{x}) d\mathbf{x}, \quad (1.1)$$

which acts as the partition function. For $A \subseteq C$,

$$Q(A) = \frac{\int_A p(\mathbf{x}) d\mathbf{x}}{\int_C p(\mathbf{x}) d\mathbf{x}}. \quad (1.2)$$

Numerical error in constrained integration can therefore propagate directly into normalisation and downstream probabilities. PAL currently uses monomial polynomial representations. Although convenient for symbolic manipulation, sampled monomial systems can become poorly conditioned as degree, dimension, or coordinate scale increases [2, 3]. Legendre and Chebyshev polynomials provide plausible alternatives because canonical-domain representations can yield better-conditioned sampled systems [8, 9, 10, 4]. Better conditioning alone, however, does not imply more accurate constrained inference: coefficient construction, scaling, arithmetic precision, integration, and accumulation also contribute numerical error [3, 11, 12].

1.2 Research Question

The study compares monomial, Legendre, and Chebyshev representations of the *same mathematical polynomials*. Holding the function fixed separates basis-dependent numerical effects from approximation differences. The central question is:

To what extent can polynomial basis choice improve the numerical stability of constrained probabilistic inference, and under which conditions do those improvements translate into more accurate PAL-relevant computations?

Four questions structure the evaluation:

- RQ1:** How do the three representations differ in conditioning, numerical rank, and direct integration accuracy?
- RQ2:** How do degree, dimension, physical scale, arithmetic precision, and coefficient perturbations affect those properties?
- RQ3:** Under constrained polytopes and numerical coefficient recovery, which errors arise from representation and which from reconstruction or integration?
- RQ4:** Do those numerical differences propagate to PAL-relevant partition functions and normalised probability queries?

Condition number is therefore treated as a diagnostic rather than sufficient evidence of numerical superiority.

1.3 Experimental Approach and Contributions

The experiments progress from controlled numerical tests to PAL-relevant inference. Analytical simplex benchmarks isolate degree, dimension, scale, precision, and coefficient perturbations. Positive polynomial densities then test partition functions and probability queries. Constrained-polytope experiments introduce non-convex and changing feasible regions together with coefficient recovery, followed by a representative PAL computation. The evaluation separates

conditioning $\rightarrow$ coefficient stability $\rightarrow$ integration accuracy $\rightarrow$ inference accuracy.
(1.3)

The dissertation contributes: (i) paired cross-basis evaluation of equivalent multivariate polynomials; (ii) conditioning, rank, perturbation, precision, scale, and dimension

analysis; (iii) constrained and downstream evaluation; and (iv) identification of coefficient construction/recovery as a stage that can erase the numerical advantages of a better-conditioned representation.

1.4 Scope and Structure

The work focuses on polynomial representation and constrained integration within PAL rather than proposing a new SMT solver, polytope algorithm, or neural architecture. Legendre and Chebyshev polynomials are tested as alternatives to monomials. Splines are not evaluated as an alternative basis because their piecewise structure and knot selection introduce additional design variables [13]. Coefficient-space perturbations are basis dependent, physical-domain scaling is distinguished from changing only the feasible region, and the representative PAL benchmark is not a complete trained neural end-to-end workload. Chapter 2 gives the required numerical background; Chapter 3 defines the experiments; Chapter 4 describes the implementation; Chapter 5 presents the evidence; Chapter 6 interprets the results; and Chapter 7 concludes. Detailed supporting figures, tables, code excerpts, and reproducibility material are retained in the appendices.

Chapter 2

Background

This chapter defines the PAL integration setting, polynomial representations, coordinate scaling, conditioning diagnostics, analytical references, and error metrics used by the experiments.

2.1 Constrained Probabilistic Inference and PAL

Let $p(\mathbf{x}) \geq 0$ be an unnormalised density on $D \subseteq \mathbb{R}^n$, with feasible region $C \subseteq D$. The constrained mass and normalised constrained density are

$$Z_C = \int_C p(\mathbf{x}) d\mathbf{x}, \quad (2.1)$$

$$p_C(\mathbf{x}) = \begin{cases} p(\mathbf{x})/Z_C, & \mathbf{x} \in C, \\ 0, & \mathbf{x} \notin C. \end{cases} \quad (2.2)$$

PAL expresses feasible regions through Satisfiability Modulo Theories over Linear Real Arithmetic (SMT(LRA)) [14, 1]. Half-spaces $\mathbf{a}_i^\top \mathbf{x} \leq b_i$ define convex polytopes, while Boolean combinations can represent non-convex or disconnected regions [15]. Weighted Model Integration (WMI) associates such logical regions with continuous weights [16, 17, 18]:

$$\text{WMI}(\phi, w) = \int_{C_\phi} w(\mathbf{x}) d\mathbf{x}. \quad (2.3)$$

PAL applies this principle to learnable piecewise-polynomial densities [1]. The numerical path relevant here is

$$\text{coefficients} \rightarrow \text{constrained integration} \rightarrow \text{partition function} \rightarrow \text{normalised probability}. \quad (2.4)$$

2.1.1 Running Constraint Example

A recurring non-convex example uses the ambient domain $D = [0, 1]^2$ and removes the centred obstacle $[0.4, 0.6]^2$:

$$\Psi_{\text{example}} = (0 \leq x \leq 1) \wedge (0 \leq y \leq 1) \wedge \neg[0.4 \leq x \leq 0.6 \wedge 0.4 \leq y \leq 0.6]. \quad (2.5)$$

The ambient polynomial domain remains fixed when only the feasible constraint

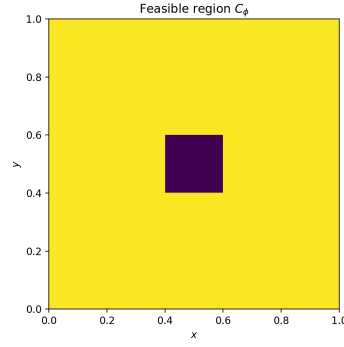


Figure 2.1: Running SMT(LRA) example: the polynomial is defined on the unit square while the central obstacle is excluded from the feasible region.

changes; this distinction is important for canonical scaling.

2.2 Polynomial Representations and Scaling

For basis functions $\{\phi_j\}_{j=1}^M$,

$$p(\mathbf{x}) = \mathbf{c}^\top \Phi(\mathbf{x}) = \sum_{j=1}^M c_j \phi_j(\mathbf{x}). \quad (2.6)$$

For dimension n and maximum total degree d , all three representations use

$$\mathcal{A}_{n,d} = \{\alpha \in \mathbb{N}_0^n : |\alpha| \leq d\}, \quad M(n,d) = \binom{n+d}{d}. \quad (2.7)$$

This combinatorial growth becomes important as dimension increases.

2.2.1 Monomial, Legendre, and Chebyshev Bases

The monomial representation is

$$p(\mathbf{x}) = \sum_{\alpha \in \mathcal{A}_{n,d}} a_\alpha \mathbf{x}^\alpha, \quad \mathbf{x}^\alpha = \prod_{r=1}^n x_r^{\alpha_r}. \quad (2.8)$$

Monomials are simple to manipulate and integrate, but powers of the input can span very different magnitudes and sampled columns can become nearly dependent [2, 3]. Legendre and first-kind Chebyshev tensor-product basis functions are

$$\Phi_{\alpha}^{(L)}(\tilde{\mathbf{x}}) = \prod_{r=1}^n P_{\alpha_r}(\tilde{x}_r), \quad (2.9)$$

$$\Phi_{\alpha}^{(C)}(\tilde{\mathbf{x}}) = \prod_{r=1}^n T_{\alpha_r}(\tilde{x}_r). \quad (2.10)$$

Legendre polynomials are orthogonal under the uniform inner product on $[-1, 1]$, while Chebyshev polynomials use weight $(1 - \tilde{x}^2)^{-1/2}$ [8, 9, 19, 10]. Their canonical orthogonality does not imply orthogonality after restriction to an arbitrary feasible region.

2.2.2 Canonical Coordinate Scaling

For $x_r \in [l_r, u_r]$, the orthogonal bases use

$$\tilde{x}_r = \frac{2x_r - (u_r + l_r)}{u_r - l_r}, \quad (2.11)$$

which maps the *ambient polynomial domain* to $[-1, 1]$ [4, 10]. It is not recomputed merely because the feasible constraint changes. For example,

$$C_s = \{\mathbf{x} \in [0, 1]^2 : x_1 \geq 0, x_2 \geq 0, x_1 + x_2 \leq s\}, \quad 0 < s \leq 1, \quad (2.12)$$

changes the feasible region while leaving the ambient polynomial coordinates fixed. This differs from experiments that deliberately scale the physical domain itself.

2.3 Conditioning Diagnostics

Two matrices measure complementary numerical effects. For evaluation points $\{\mathbf{x}^{(i)}\}_{i=1}^N$, the sampled basis matrix is

$$V_{ij} = \phi_j(\mathbf{x}^{(i)}), \quad V \in \mathbb{R}^{N \times M}. \quad (2.13)$$

For monomials this is a generalised Vandermonde system [2, 20]. Its spectral condition number and retained numerical-rank fraction are

$$\kappa_2(V) = \frac{\sigma_{\max}(V)}{\sigma_{\min}(V)}, \quad (2.14)$$

$$\rho_{\text{rank}} = \frac{\text{rank}_{\text{num}}(V)}{M}. \quad (2.15)$$

A large $\kappa_2(V)$ indicates potential amplification when recovering coefficients from $\mathbf{y} = V\mathbf{c}$ [3, 11, 12]; it does *not* imply that forward integration from already-known coefficients must be inaccurate. PAL ultimately integrates over C , motivating the constraint-induced matrix

$$G_C = \int_C \Phi(\mathbf{x})\Phi(\mathbf{x})^\top d\mathbf{x}, \quad (G_C)_{ij} = \int_C \phi_i(\mathbf{x})\phi_j(\mathbf{x}) d\mathbf{x}. \quad (2.16)$$

For

$$p_1(\mathbf{x}) = \mathbf{a}^\top \Phi(\mathbf{x}), \quad p_2(\mathbf{x}) = \mathbf{b}^\top \Phi(\mathbf{x}), \quad (2.17)$$

the constrained product integral satisfies

$$\int_C p_1(\mathbf{x})p_2(\mathbf{x}) d\mathbf{x} = \mathbf{a}^\top G_C \mathbf{b}. \quad (2.18)$$

For compatibility with later references, the individual entry definition is

$$(G_C)_{ij} = \int_C \phi_i(\mathbf{x})\phi_j(\mathbf{x}) d\mathbf{x}. \quad (2.19)$$

Thus V measures sampled distinguishability and coefficient recoverability, whereas G_C measures interaction of the basis under the actual constrained integration geometry.

2.4 Analytical Simplex Reference

The controlled experiments use the n -dimensional unit simplex

$$\Delta_n = \left\{ \mathbf{x} \in \mathbb{R}^n : x_i \geq 0, \sum_{i=1}^n x_i \leq 1 \right\}. \quad (2.20)$$

Its monomial moments are

$$\int_{\Delta_n} \mathbf{x}^\alpha d\mathbf{x} = \frac{\prod_{i=1}^n \Gamma(\alpha_i + 1)}{\Gamma\left(n + \sum_{i=1}^n \alpha_i + 1\right)}. \quad (2.21)$$

Hence, for monomial coefficients a_α ,

$$I_{\text{ref}} = \sum_{\alpha \in \mathcal{A}_{n,d}} a_\alpha \frac{\prod_{i=1}^n \Gamma(\alpha_i + 1)}{\Gamma\left(n + \sum_{i=1}^n \alpha_i + 1\right)}. \quad (2.22)$$

Every equivalent representation is compared against this common value rather than validating itself with a separate numerical quadrature routine [21, 12]. Because Gamma evaluation and accumulation are still finite-precision operations, this is described as an *analytically derived numerical reference*.

2.5 Perturbation, Error, and Positive-Density Metrics

For coefficient vector $\mathbf{c}$, a relative random perturbation is

$$\tilde{\mathbf{c}} = \mathbf{c} + \delta \max(\|\mathbf{c}\|_2, \epsilon_c) \frac{\mathbf{r}}{\|\mathbf{r}\|_2}, \quad \mathbf{r} \sim \mathcal{N}(\mathbf{0}, I), \quad (2.23)$$

with empirical integral sensitivity

$$S_\delta = \frac{|\hat{I}_\delta - \hat{I}|}{\delta \max(|\hat{I}|, \epsilon_I)}. \quad (2.24)$$

Equal coefficient perturbations are not identical function-space perturbations across bases; S_δ therefore measures representation robustness rather than a basis-independent worst-case condition measure [3, 11]. The forward, partition-function, and query errors are

$$E_I = \frac{|\hat{I} - I_{\text{ref}}|}{\max(|I_{\text{ref}}|, \epsilon_I)}, \quad (2.25)$$

$$E_Z = \frac{|\hat{Z}_C - Z_C^{\text{ref}}|}{\max(|Z_C^{\text{ref}}|, \epsilon_Z)}, \quad (2.26)$$

$$E_Q(A) = \left| \hat{Q}(A) - Q_{\text{ref}}(A) \right|. \quad (2.27)$$

The corresponding query probability is

$$Q(A) = \frac{\int_A p(\mathbf{x}) d\mathbf{x}}{\int_C p(\mathbf{x}) d\mathbf{x}}. \quad (2.28)$$

Probability experiments require non-negative densities. A sampled source polynomial q is therefore transformed into

$$p(\mathbf{x}) = q(\mathbf{x})^2 + \eta, \quad \eta > 0, \quad (2.29)$$

so that $p(\mathbf{x}) \geq \eta$.

2.6 Representation and Pipeline Stability

The diagnostics above describe different numerical stages. A large $\kappa_2(V)$ can make coefficient recovery fragile while forward integration from accurate coefficients remains stable. A basis may retain full rank in `float64` but lose directions in `float32`; likewise,

a well-conditioned target basis can perform poorly if its coefficients are constructed through an unstable conversion. The relevant pipeline is therefore

representation $\rightarrow$ coefficient construction/recovery $\rightarrow$ constrained integration $\rightarrow$ normalisation.
(2.30)

This distinction motivates the experimental design in Chapter 3.

Chapter 3

Methodology

This chapter defines the experiments used to determine when basis-dependent conditioning affects constrained probabilistic inference. The design separates representation conditioning, coefficient construction/recovery, constrained integration, and downstream probability computation so that numerical failures are attributed to the stage that produces them.

3.1 Research Questions and Common Design

The evaluation addresses four questions:

- RQ1:** How does the conditioning of monomial, Legendre, and Chebyshev representations change with degree, dimension, geometric scale, and arithmetic precision?
- RQ2:** Do conditioning differences produce measurable differences in coefficient recovery, perturbation sensitivity, and constrained integration error?
- RQ3:** Do these effects persist for non-trivial and changing SMT(LRA)-style constraints?
- RQ4:** Do representation-level numerical differences propagate to PAL-relevant partition functions and normalised query probabilities?

All cross-basis comparisons represent the same mathematical polynomial and, within a configuration, share feasible geometry, evaluation points, precision, perturbation magnitude, and random seed. Only the polynomial coordinate representation changes:

$$\mathcal{B} \in \{\text{monomial}, \text{Legendre}, \text{Chebyshev}\}. \quad (3.1)$$

Legendre and Chebyshev coordinates use the canonical map in Equation (2.11). Except

when physical scale itself is varied, that map is determined by the ambient polynomial domain rather than by the current feasible region.

3.1.1 Equivalent Polynomial Construction

A source polynomial

$$p(\mathbf{x}) = \sum_{\alpha \in \mathcal{A}_{n,d}} a_{\alpha} \mathbf{x}^{\alpha} \quad (3.2)$$

is converted so that

$$p_{\text{mono}}(\mathbf{x}) = p_{\text{leg}}(\mathbf{x}) = p_{\text{cheb}}(\mathbf{x}) \quad (3.3)$$

up to numerical conversion tolerance. Equivalence is checked on independent validation points using

$$E_{\text{repr}}^{(\mathcal{B})} = \frac{\|p_{\mathcal{B}}(\mathbf{X}_{\text{val}}) - p_{\text{src}}(\mathbf{X}_{\text{val}})\|_2}{\max(\|p_{\text{src}}(\mathbf{X}_{\text{val}})\|_2, \epsilon)}. \quad (3.4)$$

A downstream discrepancy is not interpreted as a basis effect if this equivalence check already fails substantially.

3.1.2 Coefficient Scaling

Random source coefficients are degree-scaled to avoid trivial domination by coefficient growth:

$$s_{\alpha} = \frac{s_0}{1 + |\alpha|}. \quad (3.5)$$

For downstream positive-density experiments,

$$s_0 = 0.35, \quad (3.6)$$

before constructing $p = q^2 + \eta$.

3.2 Conditioning, Scale, Dimension, and Recovery

The first group of experiments isolates representation-level behaviour before testing downstream inference.

3.2.1 Sampled Conditioning Benchmarks

For common points $\mathbf{x}^{(i)}$, each basis constructs

$$V_{ij}^{(\mathcal{B})} = \phi_j^{(\mathcal{B})}(\mathbf{x}^{(i)}), \quad (3.7)$$

from which $\kappa_2(V)$ and the retained numerical-rank fraction are recorded. Three controlled variations are used:

- (i) **Degree:** a two-dimensional unit simplex in `float64`, varying polynomial degree.
- (ii) **Physical scale:** $n = 2, d = 10$, while dilating the ambient physical simplex and changing the canonical map consistently.
- (iii) **Dimension:**

$$n \in \{1, 2, 3, 4, 5\}, \quad M(n, d) = \binom{n+d}{d},$$

to measure how combinatorial basis growth affects conditioning and rank.

For compatibility with later references, the fixed scale configuration is

$$n = 2, \quad d = 10, \quad (3.8)$$

and the tested dimensions are

$$n \in \{1, 2, 3, 4, 5\}. \quad (3.9)$$

3.2.2 Constraint-Induced Conditioning

Sampled conditioning does not directly measure interaction under constrained integration. A complementary benchmark therefore constructs

$$G_C^{(\mathcal{B})} = \int_C \Phi^{(\mathcal{B})}(\mathbf{x}) \Phi^{(\mathcal{B})}(\mathbf{x})^\top d\mathbf{x}. \quad (3.10)$$

The completed experiment uses

$$n = 2, \quad d \in \{0, 1, 2, 3, 5, 6, 8, 10\}, \quad (3.11)$$

on the unit simplex and the unit square with a centred square obstacle. The ambient orthogonal map remains

$$\xi = 2x - 1. \quad (3.12)$$

Products of basis functions are converted to physical-coordinate monomials and integrated using analytical moments. Symmetry, $(G_C)_{ij} = (G_C)_{ji}$, allows only one triangular half to be evaluated explicitly. If a singular direction falls below the numerical-rank criterion, the configuration is retained as

$$\kappa_2(G_C) = \infty \quad (3.13)$$

rather than being assigned a misleading finite condition number.

3.2.3 Coefficient Recovery

Conditioning becomes operational when coefficients must be reconstructed from sampled values. For

$$V^{(\mathcal{B})}\hat{\mathbf{c}}^{(\mathcal{B})} \approx \mathbf{y}, \quad (3.14)$$

coefficients are recovered by least squares rather than explicit inversion. Relative coefficient error is

$$E_c^{(\mathcal{B})} = \frac{\|\hat{\mathbf{c}}^{(\mathcal{B})} - \mathbf{c}^{(\mathcal{B})}\|_2}{\max(\|\mathbf{c}^{(\mathcal{B})}\|_2, \epsilon_c)}. \quad (3.15)$$

The same V is used for both recovery and its reported condition number, so the diagnostic refers to the numerical problem actually solved.

3.3 Precision, Perturbation, and Simplex Integration

Reduced precision is tested in

$$n \in \{1, 2, 3\}, \quad \text{dtype} \in \{\text{float32}, \text{float64}\}, \quad (3.16)$$

with ten trials per configuration. Relative coefficient perturbations use

$$\delta = 10^{-5}, \quad (3.17)$$

and the perturbation in Equation (2.23). Condition number, numerical rank, direct integration error, and S_δ are recorded. The controlled simplex experiments compare every representation with the single common analytical reference in Equation (2.22).

For basis $\mathcal{B}$,

$$E_I^{(\mathcal{B})} = \frac{|\hat{I}^{(\mathcal{B})} - I_{\text{ref}}|}{\max(|I_{\text{ref}}|, \epsilon_I)}. \quad (3.18)$$

This prevents each representation from validating itself with its own integration routine.

3.4 Downstream Probability Experiment

To test whether representation-level effects reach probabilistic quantities, a source polynomial

$$q(\mathbf{x}) = \sum_{\alpha} b_{\alpha} \mathbf{x}^{\alpha} \quad (3.19)$$

is transformed into

$$p(\mathbf{x}) = q(\mathbf{x})^2 + \eta, \quad \eta = 0.1. \quad (3.20)$$

The configurations are

$$d_q \in \{0, 1, 2, 3, 5, 8\}, \quad (3.21)$$

$$d_p \in \{0, 2, 4, 6, 10, 16\}, \quad (3.22)$$

$$n \in \{1, 2, 3\}, \quad (3.23)$$

$$s \in \{0.25, 0.50, 0.75\}, \quad (3.24)$$

where s controls the nested query region. Both `float32` and `float64` are tested with ten trials per configuration. For each basis,

$$\widehat{Z}^{(\mathcal{B})} = \widehat{I}_C^{(\mathcal{B})}, \quad (3.25)$$

$$\widehat{Q}^{(\mathcal{B})}(A) = \frac{\widehat{I}_A^{(\mathcal{B})}}{\widehat{Z}^{(\mathcal{B})}}. \quad (3.26)$$

Analytical simplex references provide Z_{ref} and $Q_{\text{ref}}(A)$. Partition error, absolute query error, perturbation sensitivity, and runtime are retained.

3.5 Constrained-Polytope Experiments

The next experiments replace simplex geometry with SMT(LRA)-style box-with-obstacle domains. The completed main sweep uses

$$n \in \{2, 3\}, \quad d_q \in \{0, 1, 2, 3, 5\}, \quad d_p \in \{0, 2, 4, 6, 10\}, \quad (3.27)$$

where $d_p = 2d_q$ because the positive density is formed as $p(\mathbf{x}) = q(\mathbf{x})^2 + \eta$. Thus the constrained-polytope benchmark contains no dimension-4 or density-degree-12 configuration. The same physical constraint is used for all basis representations, and analytical box subtraction provides the reference where available.

3.5.1 Static and Changing Constraints

The static experiment fixes obstacle geometry while recording conditioning, rank, direct integration error, perturbation sensitivity, equivalence diagnostics, and runtime. The dynamic experiment changes obstacle parameters while keeping the ambient polynomial domain fixed. Five schedules are used:

$$S = \{\text{shrinking, expanding, oscillating, pulsed, random walk}\}. \quad (3.28)$$

At step t ,

$$I_t = \int_{C_t} p(\mathbf{x}) d\mathbf{x}. \quad (3.29)$$

Only C_t changes; the orthogonal-coordinate map remains tied to the fixed ambient domain. This isolates changing constraints from changing polynomial coordinates.

3.5.2 Noisy Coefficient Recovery

The constrained benchmark additionally reconstructs coefficients from sampled function values. Relative value noise is applied before solving an oversampled least-squares system. The completed experiment uses oversampling factor 2.0 and relative value noise 10^{-5} . Condition number, retained rank, coefficient error, noise amplification, known-coefficient integration error, and recovered-integral error are stored separately so that recovery errors are not mistaken for failures of direct integration.

3.6 Representative PAL Evaluation

The final controlled benchmark exercises the numerical stages most directly related to PAL. It uses

$$n \in \{2, 3\}, \quad d_q \in \{0, 1, 2, 3, 5\}, \quad (3.30)$$

with ten trials per configuration. Each trial constructs one positive density, forms equivalent representations, verifies equivalence, evaluates conditioning and numerical rank, performs constrained integration, computes the partition function and a nested query probability, and records perturbation sensitivity and runtime. The resulting path is

$$\boxed{\mathbf{c} \rightarrow p(\mathbf{x}) \rightarrow \int_C p \rightarrow Z_C \rightarrow \frac{\int_A p}{Z_C}}, \quad A \subseteq C. \quad (3.31)$$

This benchmark represents the coefficient and constrained-inference stages relevant to the research question; it is not presented as a complete trained neural PAL workload.

3.6.1 Supplementary PAL Precision Stress Case

A minimal PAL/WMI reproducer supplied during project supervision is evaluated separately as a PAL-originated precision stress case. It fixes one triangular feasible region and the coefficients of a degree-12 monomial product arising from a squared spline cell, changing only whether the relevant monomial-function evaluation follows the default lower-precision path or remains in `float64`. Because the integrand is mathematically non-negative, preservation of the integral's non-negative sign is used as a qualitative correctness criterion. The `float64` value is not treated as arbitrary-precision

ground truth. No cross-basis conclusion is drawn: a valid Legendre or Chebyshev extension would have to preserve the same triangle, mathematical polynomial, and coefficient-construction semantics while changing only the representation.

3.7 Controls, Runtime, and Interpretation

Runtime includes the numerical work required by each basis-specific path and is summarised across repeated trials. It is interpreted jointly with numerical accuracy rather than as an independent measure of superiority. All experiments use deterministic seeds and paired configurations. Within a comparison the bases share the underlying polynomial, sample points, feasible geometry, precision, perturbation magnitude, and trial seed. Row-level results are stored before plotting, with configuration summaries and final reproducibility information reported in Appendix E. Four rules govern interpretation:

1. A smaller condition number is diagnostic of reduced sensitivity, not proof of smaller integration error.
2. Cross-basis conclusions require numerical equivalence of the represented functions.
3. Conversion and recovery errors are treated separately from representation and integration errors.
4. Partition-function and normalised-query errors provide the strongest evidence of practical PAL relevance.

These controls prevent canonical orthogonality from being interpreted as automatic superiority under constrained probabilistic inference.

Chapter 4

Implementation

4.1 Overview

The implementation extends the existing PAL codebase through an experimental numerical layer while preserving the original monomial path as the baseline. Shared utilities provide Legendre and Chebyshev evaluation, basis conversion, conditioning diagnostics, analytical integration, perturbation, coefficient recovery, and downstream probability evaluation. The complete implementation, benchmark runners, plotting utilities, and reproducibility material are maintained in the project repository: <https://github.com/k-r-o-y/pal>. The software separates row-level benchmark execution from plotting and aggregation so that numerical results can be traced to their generating configurations. Exact commands, result artefacts, and environment information are reported in Appendix E.

4.2 Basis Representation and Conversion

Multivariate polynomials use the common total-degree index set from Equation (2.7). The same ordering is reused for coefficient vectors, basis matrices, conversion matrices, and integration weights; inconsistent ordering would produce numerically valid operations for the wrong polynomial. For basis $\mathcal{B}$, conversion to physical-coordinate monomial coefficients is written

$$\mathbf{a} = T_{\mathcal{B}} \mathbf{c}_{\mathcal{B}}, \quad (4.1)$$

where $\mathbf{a}$ is the monomial coefficient vector. Legendre and Chebyshev basis functions are first composed with the canonical map in Equation (2.11), expanded in physical

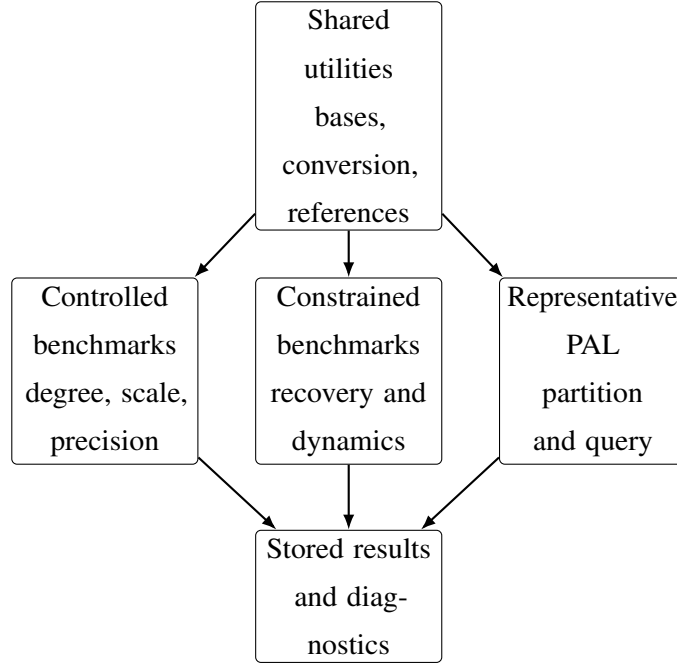


Figure 4.1: Experimental software structure. Shared numerical utilities support the controlled, constrained, and representative PAL benchmarks.

coordinates, and then assembled according to the shared multi-index order. Conversion is checked on sampled values through

$$R_{\text{basis}} = \frac{\|\mathbf{p}_{\text{converted}} - \mathbf{p}_{\text{source}}\|_2}{\max(\|\mathbf{p}_{\text{source}}\|_2, \epsilon)}. \quad (4.2)$$

This diagnostic is retained wherever conversion stability affects downstream interpretation.

4.3 Conditioning and Constraint-Induced Geometry

All basis families use the same sampled point set to construct V from Equation (2.13). Condition numbers are obtained from singular values rather than an explicit matrix inverse [3, 11], and numerical rank is computed from the same spectrum using a precision-appropriate tolerance. The coefficient-recovery experiments reuse this same matrix V , so the reported $\kappa_2(V)$ describes the system from which the coefficients are actually reconstructed. For constrained conditioning, the implementation constructs

$$G_C = \int_C \Phi(\mathbf{x}) \Phi(\mathbf{x})^\top d\mathbf{x}, \quad (4.3)$$

with entries

$$(G_C)_{ij} = I_C[\phi_i \phi_j]. \quad (4.4)$$

For the two-dimensional simplex and box-with-obstacle cases, basis products are expanded into physical-coordinate monomials and integrated using analytical moments. Matrix symmetry is exploited so that only one triangular half is evaluated explicitly. Numerically unresolved configurations are retained with $\kappa_2(G_C) = \infty$ rather than replaced by arbitrary finite estimates.

4.4 Analytical Integration

Simplex integration implements the monomial moment from Equation (2.21). To avoid direct construction of large factorials, moments are evaluated through logarithmic Gamma functions:

$$\log I_\alpha = \sum_{r=1}^n \log \Gamma(\alpha_r + 1) - \log \Gamma(n + |\alpha| + 1). \quad (4.5)$$

For a simplex scaled by s , the corresponding moment is multiplied by

$$s^{n+|\alpha|}. \quad (4.6)$$

The one-dimensional case is checked independently against ordinary interval integration before the same implementation is used in higher dimensions. Axis-aligned boxes use products of one-dimensional power moments. For the box-with-obstacle domain,

$$I_C = I_{\text{outer}} - I_{\text{obstacle}}, \quad (4.7)$$

which provides an analytical reference for the constrained-polytope benchmark without requiring generic polytope cubature.

4.5 Sampling, Perturbation, and Density Construction

Point sets are generated independently of basis choice and reused across all representations. Higher-dimensional simplex samples are generated in barycentric coordinates before conversion to explicit Cartesian coordinates. Coefficient perturbations implement Equation (2.23). Random directions are generated in the arithmetic type under test, normalised, and scaled to the requested relative magnitude. Seeds are recorded so that perturbations can be replayed. For downstream probability experiments,

$$b_\alpha \sim \mathcal{N}\left(0, \left[\frac{0.35}{1 + |\alpha|}\right]^2\right), \quad (4.8)$$

and the sampled source polynomial is transformed into the positive density $p(\mathbf{x}) = q(\mathbf{x})^2 + \eta$. Polynomial squaring is implemented by sparse multi-index convolution, combining identical resulting exponents before the constant offset is added.

4.6 Downstream Probability Computation

For basis $\mathcal{B}$, integration over a simplex of scale s is written

$$I_s = \mathbf{w}_{\mathcal{B}}(s)^{\top} \mathbf{c}_{\mathcal{B}}. \quad (4.9)$$

Monomial weights are the analytical scaled-simplex moments. Orthogonal-basis weights are obtained from the same basis-to-monomial transformation:

$$\mathbf{w}_{\mathcal{B}}^{\top} = \mathbf{w}_M^{\top} T_{\mathcal{B}}. \quad (4.10)$$

The whole-region integral gives the partition function and a nested subregion gives the query numerator. The implementation then forms the normalised query, recording partition error, query error, perturbation sensitivity, and runtime for each configuration.

4.7 Constrained Domains and Coefficient Recovery

The constrained-polytope runner reuses the same density and basis infrastructure while changing only the feasible region. It uses dimensions 2 and 3, source degrees 0, 1, 2, 3, 5, and the corresponding density degrees 0, 2, 4, 6, 10. Static experiments use a fixed obstacle; dynamic experiments vary obstacle size according to the schedules defined in Equation (3.28), while keeping the ambient polynomial domain unchanged. Coefficient recovery introduces the inverse problem

$$\hat{\mathbf{c}} = \arg \min_{\mathbf{z}} \|\mathbf{V}\mathbf{z} - \tilde{\mathbf{y}}\|_2, \quad (4.11)$$

where relative noise is added to sampled values before solving the oversampled least-squares problem. The completed benchmark uses oversampling factor 2.0 and relative function-value noise 10^{-5} . Recovered coefficients are integrated over the same constraint. Condition number, retained rank, coefficient error, noise amplification, direct known-coefficient error, and recovered-integral error are stored separately, allowing errors introduced during recovery to be distinguished from errors in forward integration.

4.8 Representative PAL Path

The representative PAL runner preserves the original monomial route while reusing the validated density, conversion, conditioning, recovery, and integration components for the experimental alternatives. Its numerical path is

$$\text{source density} \rightarrow \text{representation} \rightarrow \text{coefficient construction/recovery} \rightarrow \text{constrained integration} \rightarrow \text{partitioning} \quad (4.12)$$

For each configuration, the runner records representation-equivalence and recovery residuals, sampled-matrix condition number, numerical rank, scaling parameters, partition error, normalised-query error, perturbation sensitivity, and runtime. The same sampled system is used for conditioning and recovery, allowing a downstream failure to be connected directly with the coefficient system that produced it. The runner is therefore representative of the PAL numerical stages studied here, but not of every neural training or production-inference stage.

4.9 Validation and Reproducibility

Validation focuses on failures that could otherwise be mistaken for basis effects. The implementation checks that:

1. converted representations reproduce the source polynomial;
2. physical coordinates map correctly to the canonical domain;
3. one-dimensional simplex integration matches interval integration;
4. simplex and box moments agree with independently verifiable cases;
5. coefficient, basis-matrix, conversion, and integration-weight indices are consistent;
6. generated densities and query probabilities remain finite and valid;
7. dynamic obstacles remain inside the fixed ambient domain;
8. the constrained-polytope results and regenerated degree plots are consistent with dimensions 2, 3 and density degrees 0, 2, 4, 6, 10, with no dimension-4 or density-degree-12 polytope configuration;
9. recovery residuals remain finite and deterministic under recorded seeds; and
10. the original monomial PAL path continues to operate independently of the experimental alternatives.

The G_C implementation is additionally checked for symmetry, low-degree analytical agreement, and positive semidefiniteness up to floating-point tolerance. Numerically singular configurations remain in the result set rather than being discarded. Benchmark rows are written before aggregation and plotting. Exact commands, software versions, result filenames, and selected implementation excerpts are reported in Appendix E. The complete implementation is available at <https://github.com/k-r-o-y/pal>.

Chapter 5

Experimental Evaluation

5.1 Experimental Setup and Validation

The experiments progress from controlled polynomial benchmarks to PAL-relevant constrained inference. Repeated configurations are summarised by the median and interquartile range unless stated otherwise. Table 5.1 records the principal completed runs; full row-level outputs and supplementary tables are retained in the appendices. Before

Table 5.1: Principal completed experimental configurations.

Experiment	Configuration	Primary purpose
Degree / G_C	$n = 2$, degree up to 10	Sampled and constraint-induced conditioning.
Physical scale	$n = 2, d = 10$	Coordinate-scale sensitivity.
Dimension	$n = 1, \dots, 5$	Conditioning, forward error, and basis growth.
Precision	$n = 1, 2, 3, 1200$ float32/float64 error and rank loss. records	
Downstream	$n = 1, 2, 3, 3240$ Partition and normalised-query error. records	
Polytope	$n = 2, 3, d_p =$ Constraint geometry and coefficient recovery. $0, 2, 4, 6, 10, 37200$ records	
Representative PAL	$n = 2, 3, 600$ records	PAL-relevant coefficient and inference pipeline.

interpreting basis differences, the implementation was checked for cross-basis equivalence, canonical mapping, analytical simplex moments, one-dimensional agreement, deterministic seeds, density validity, partition/query consistency, constrained-domain

execution, and finite outputs. As a compact sanity check, the degree-5 one-dimensional median condition numbers are approximately

$$2.33 \times 10^4, \quad 4.45 \times 10^1, \quad 5.55 \times 10^1$$

for monomial, Legendre, and Chebyshev representations, respectively, while direct integration remains at double-precision rounding scale.

5.2 Conditioning with Degree and Constraint Geometry

The controlled degree experiment shows a clear representation effect before a corresponding forward-integration effect appears. At degree 5, median sampled condition

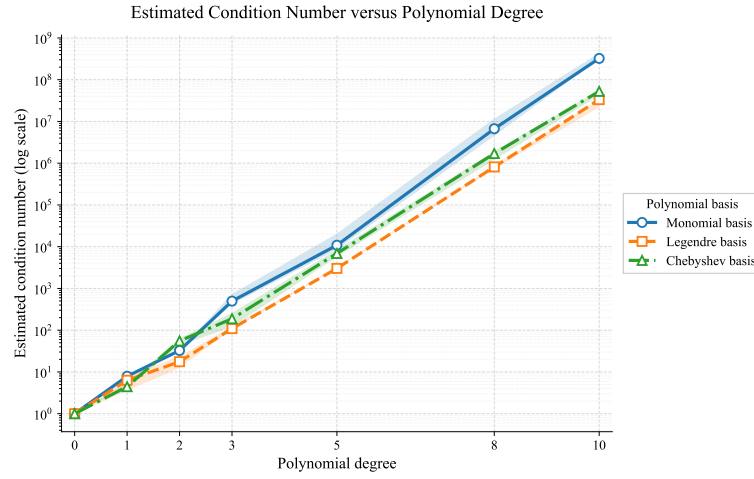


Figure 5.1: Median sampled-system condition number versus polynomial degree in the two-dimensional unit-simplex benchmark.

numbers are approximately 1.08×10^4 , 3.02×10^3 , and 6.89×10^3 for monomial, Legendre, and Chebyshev representations. At degree 10, they increase to

$$3.24 \times 10^8, \quad 3.33 \times 10^7, \quad 5.25 \times 10^7,$$

so the monomial sampled system is roughly one order of magnitude more ill-conditioned than the alternatives in the highest-degree configuration. The constraint-induced matrix G_C strengthens this result by evaluating basis interaction under the feasible-region integration geometry. On the unit simplex, monomial G_C reaches approximately 1.73×10^{11} at degree 6 and becomes numerically singular at degree 8. At degree 8, Legendre and Chebyshev remain finite at approximately 2.06×10^{11} and 3.96×10^{11} , respectively.

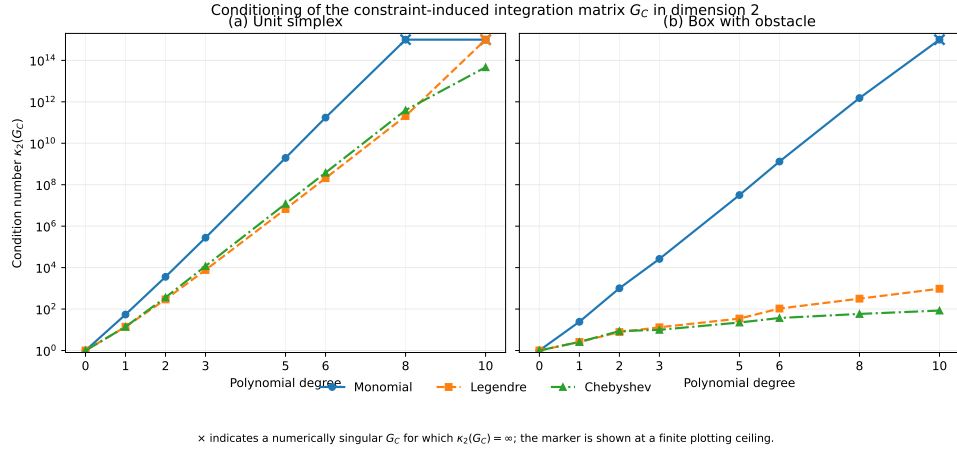


Figure 5.2: Condition number of G_C versus degree for the unit simplex and box-with-obstacle region. Cross markers denote numerically singular cases.

At degree 10, Legendre also becomes numerically singular, while Chebyshev remains finite at approximately 4.66×10^{13} . The box-with-obstacle region produces a much larger separation. At degree 8,

$$\kappa_2(G_C) \approx 1.52 \times 10^{12}, \quad 3.17 \times 10^2, \quad 5.82 \times 10^1$$

for monomial, Legendre, and Chebyshev bases. At degree 10, the monomial matrix is numerically singular, while Legendre and Chebyshev remain full rank with condition numbers approximately 9.56×10^2 and 8.44×10^1 . The corresponding rank data confirm that these infinite values represent actual numerical-rank loss. On the simplex, the monomial retained fraction is approximately 0.956 at degree 8 and 0.788 at degree 10; Legendre retains approximately 0.985 at degree 10, and Chebyshev remains full rank. In the box-with-obstacle case, monomials retain approximately 0.879 at degree 10, while both orthogonal bases remain full rank. Despite these conditioning differences, direct unit-simplex integration is still numerically benign. At degree 10, median relative errors are about

$$5.54 \times 10^{-16}, \quad 3.82 \times 10^{-16}, \quad 5.54 \times 10^{-16},$$

for monomial, Legendre, and Chebyshev. Thus poor sampled conditioning does not by itself imply inaccurate forward integration from already-available coefficients.

5.3 Physical Scale and Dimension

Physical scaling reveals sensitivity that is largely hidden in the benign unit-scale integral. At scale 1, median sampled condition numbers are approximately

$$2.41 \times 10^8, \quad 2.62 \times 10^7, \quad 6.85 \times 10^7,$$

for monomial, Legendre, and Chebyshev. At scale 100, the corresponding values are approximately

$$3.03 \times 10^8, \quad 2.57 \times 10^7, \quad 5.23 \times 10^7.$$

The larger practical difference appears in coefficient-perturbation sensitivity. Representative monomial values rise to approximately 1.18×10^3 at scale 3, 3.58×10^9 at scale 10, and 1.29×10^{21} at scale 100, whereas the orthogonal-basis values remain many orders of magnitude smaller. Large physical scale therefore turns the conditioning distinction into a substantial robustness difference. The dimension benchmark gives a

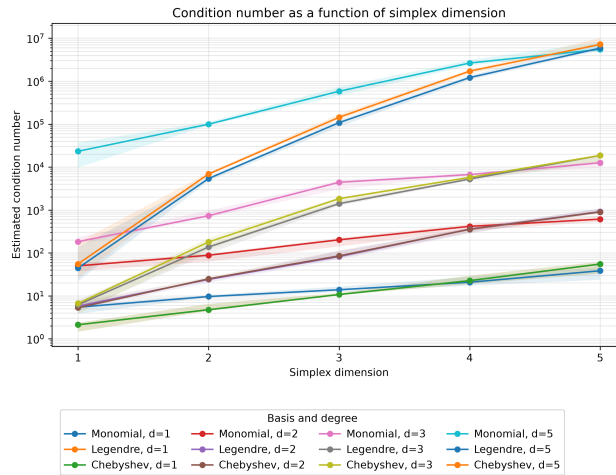


Figure 5.3: Median sampled-system condition number as simplex dimension increases.

more qualified picture. At degree 5,

n	Monomial	Legendre	Chebyshev
1	2.33×10^4	4.45×10^1	5.55×10^1
3	5.82×10^5	1.08×10^5	1.44×10^5
5	5.50×10^6	5.88×10^6	7.18×10^6

The orthogonal advantage is strongest at lower dimension and higher degree; by dimension 5 at degree 5, all three systems are of the same order. Direct float64 integration

nevertheless remains accurate: at degree 5, median Legendre error grows from approximately 1.49×10^{-16} in one dimension to 2.49×10^{-15} in five, while Chebyshev reaches approximately 1.06×10^{-15} . The basis count grows from 6 at $n = 1, d = 5$ to 252 at $n = 5, d = 5$. The experimental orthogonal paths are correspondingly slower because they include the current conversion and product-basis machinery; these timings are not treated as inherent costs of the polynomial families.

5.4 Reduced Precision

Reduced precision converts the conditioning differences into clearer finite-precision consequences. Across the 1200-record benchmark, median `float32` relative integration

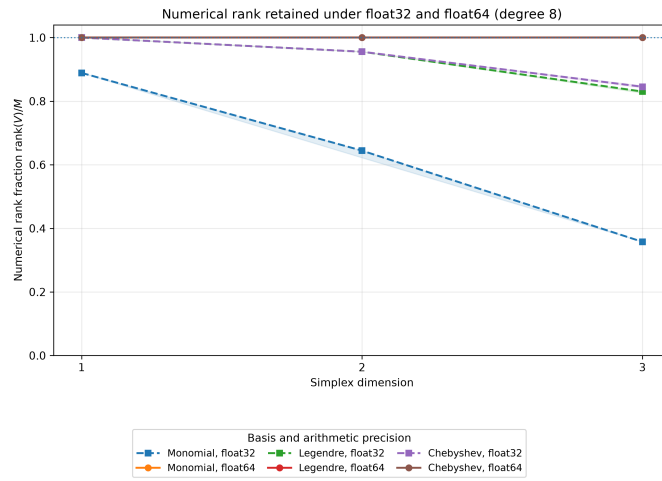


Figure 5.4: Retained numerical-rank fraction at polynomial degree 8 under `float32` as simplex dimension increases.

error is approximately 5.92×10^{-8} , with first and third quartiles 2.54×10^{-8} and 1.09×10^{-7} , and maximum 9.21×10^{-7} . The `float64` median is recorded as zero, its upper quartile is approximately 2.12×10^{-16} , and its maximum is approximately 9.54×10^{-15} . The stronger distinction is numerical rank. All tested `float64` systems remain full rank. At degree 8, the `float32` monomial rank fraction decreases from approximately 0.89 in one dimension to 0.64 in two dimensions and 0.35 in three. The orthogonal systems retain full rank in one dimension, approximately 0.96 in two, and approximately 0.82–0.85 in three. Thus considerably more orthogonal basis directions remain numerically distinguishable in reduced precision.

5.5 Downstream Probability Evaluation

The downstream benchmark uses strictly positive densities $p(\mathbf{x}) = q(\mathbf{x})^2 + 0.1$. The completed experiment contains 3240 finite records; no generated density is negative and no non-finite partition or query result is produced. Representative density visualisations and additional equivalence checks are retained in Appendix B. The dominant separation

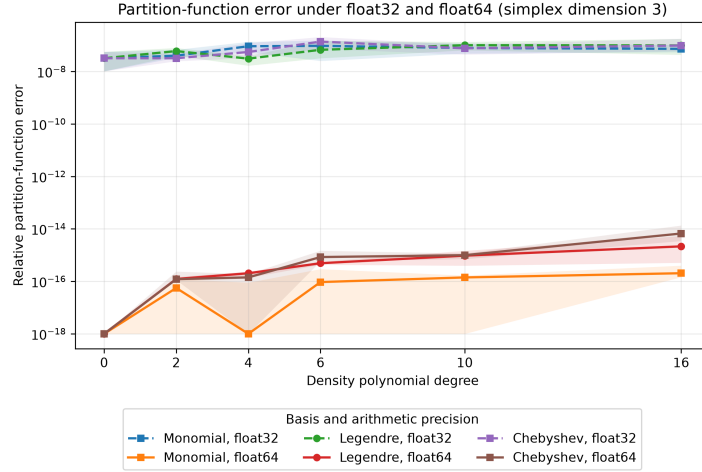


Figure 5.5: Partition-function relative error versus density degree under `float32` and `float64`, shown for dimension 3.

is arithmetic precision. Aggregate median `float32` partition errors are approximately

$$4.88 \times 10^{-8} \quad (\text{monomial}), \quad 2.91 \times 10^{-8} \quad (\text{Legendre}), \quad 3.97 \times 10^{-8} \quad (\text{Chebyshev}).$$

The corresponding `float64` medians are zero for the direct monomial path and approximately 1.56×10^{-16} and 1.60×10^{-16} for Legendre and Chebyshev. Median `float32` absolute query errors are approximately 9.92×10^{-9} , 9.83×10^{-9} , and 1.04×10^{-8} for monomial, Legendre, and Chebyshev, respectively. The `float64` medians are 3.47×10^{-18} for monomials and 2.78×10^{-17} for both orthogonal representations. The largest observed absolute query error is approximately 7.17×10^{-7} , from a `float32` Chebyshev configuration at the largest tested degree. These results are therefore precision dominated: within either arithmetic type, basis-dependent probability errors are much more similar than the conditioning differences alone would suggest. Runtime shows the opposite trade-off in the current implementation. Aggregate monomial medians are approximately 0.0473 ms in `float32` and 0.0402 ms in `float64`; Legendre is approximately 2.37 and 2.34 ms, and Chebyshev 2.15 and 2.17 ms. These values include conversion and product-basis overhead and are not intrinsic complexity bounds.

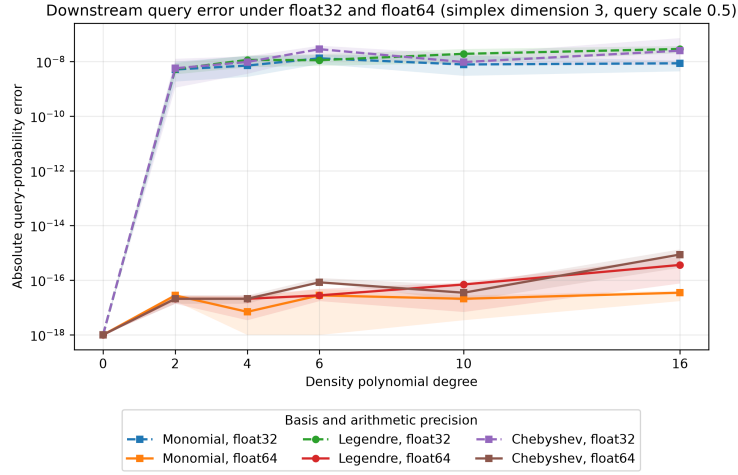


Figure 5.6: Absolute normalised query-probability error versus density degree for dimension 3 and query scale 0.50.

5.6 Constrained Polytope and Coefficient Recovery

The constrained-polytope benchmark introduces non-convex feasible regions, changing constraints, and numerical coefficient recovery. The completed run contains 37200 records over dimensions 2 and 3, source degrees 0,1,2,3,5, and density degrees 0,2,4,6,10. The degree-dependent figures in this section use the dimension-3 dynamic box-with-obstacle configuration and aggregate the rows across the five completed dynamic schedules, twelve trajectory steps, and ten trials at each degree using the median and interquartile range. This is the same configuration family defined in Section 3.5. At high degree, monomial sampled systems are substantially more ill-conditioned than the alternatives. In the dimension-3 dynamic box-with-obstacle benchmark, the largest tested monomial condition numbers are around 10^8 , compared with approximately 10^3 or below for the orthogonal representations. The `float32` monomial system begins losing numerical rank at density degree 6 and retains only approximately

$$\rho_{\text{rank}} \approx 0.36$$

at degree 10; Legendre, Chebyshev, and the `float64` monomial systems remain full rank over the tested range. Recovery makes the conditioning difference operational. Monomial noise amplification grows from approximately order one at degree 0 to approximately 10^6 at density degree 10, whereas Legendre and Chebyshev remain around order 10^1 . Despite this large recovery-sensitivity difference, recovered-integral errors cluster around 10^{-6} – 10^{-5} across representations, with substantial overlap in

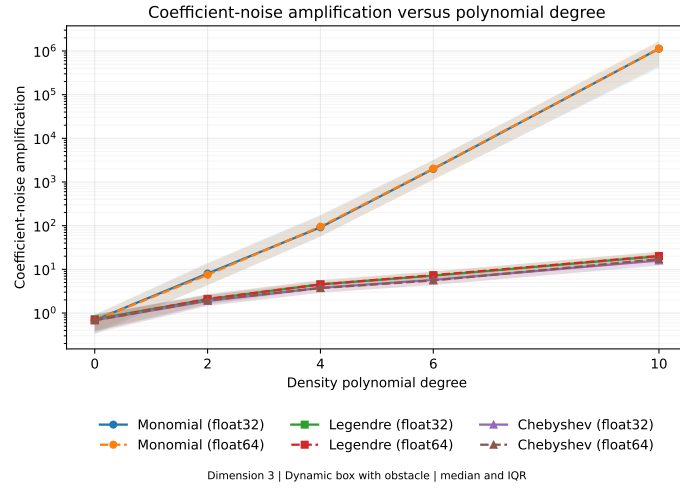


Figure 5.7: Coefficient-noise amplification versus density degree for the dimension-3 dynamic box-with-obstacle benchmark. Curves show the median and interquartile range over all five dynamic schedules, twelve trajectory steps, and ten trials for each degree.

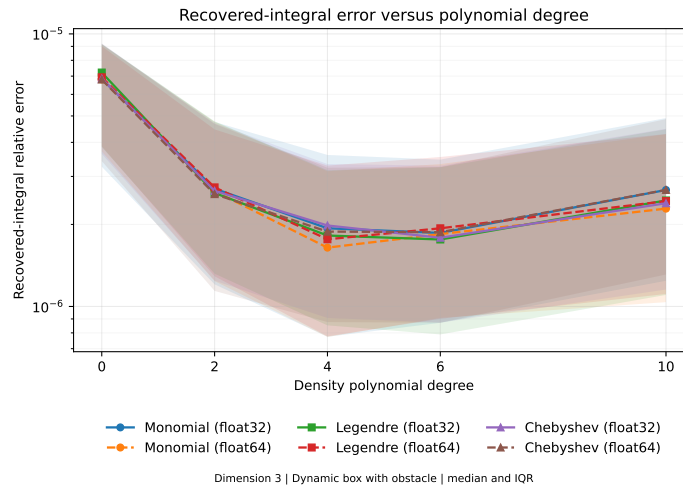


Figure 5.8: Recovered constrained-integral relative error versus density degree for the dimension-3 dynamic box-with-obstacle benchmark. Curves show the median and interquartile range over all five dynamic schedules, twelve trajectory steps, and ten trials for each degree.

the interquartile ranges. Improving coefficient recoverability therefore does not automatically improve the final integral if another reconstruction or integration stage dominates. Changing-constraint schedules produce similar direct-error levels within each basis–precision pair. Direct `float32` errors remain approximately 10^{-8} – 10^{-7} , orthogonal `float64` paths around 10^{-14} , and the direct `float64` monomial path near machine precision. No catastrophic spike occurs in the tested trajectories, so changing obstacle geometry itself is not the dominant numerical failure mode here. Runtime grows markedly with dimension and is larger for the current orthogonal implementation, but again includes conversion, product-basis, and recovery overhead rather than measuring an intrinsic basis cost.

5.7 Representative PAL Evaluation

The representative PAL benchmark contains 600 finite records across dimensions 2 and 3, source degrees 0, 1, 2, 3, 5, corresponding density degrees 0, 2, 4, 6, 10, all three bases, both precisions, and ten trials.

5.7.1 Conditioning and Downstream Error

The associated coefficient systems are substantially more ill-conditioned than the controlled simplex systems. Beyond the constant-density case, condition numbers are generally between approximately 10^{16} and 10^{18} , with one monomial density-degree-2 configuration exceeding 10^{25} . The near-overlap of the precision curves indicates that this conditioning is mainly a property of the coefficient-construction system, rather than merely storage type. The direct monomial `float32` path remains around order 10^{-8} , while the `float64` monomial path remains near machine precision. The current Legendre and Chebyshev paths, however, develop approximately order-one partition-function error from density degree 2. This is not interpreted as an intrinsic failure of orthogonal polynomials: the controlled experiments show accurate orthogonal forward integration. Instead, the PAL benchmark indicates that the current coefficient conversion/recovery procedure fails to preserve the represented polynomial in these highly ill-conditioned systems. The upstream failure propagates into normalised inference. Monomial `float32` query error remains around 10^{-8} , with `float64` near machine precision, whereas the current Legendre and Chebyshev paths produce approximately 10^{-2} query error from density degree 2. Normalisation therefore does not remove errors

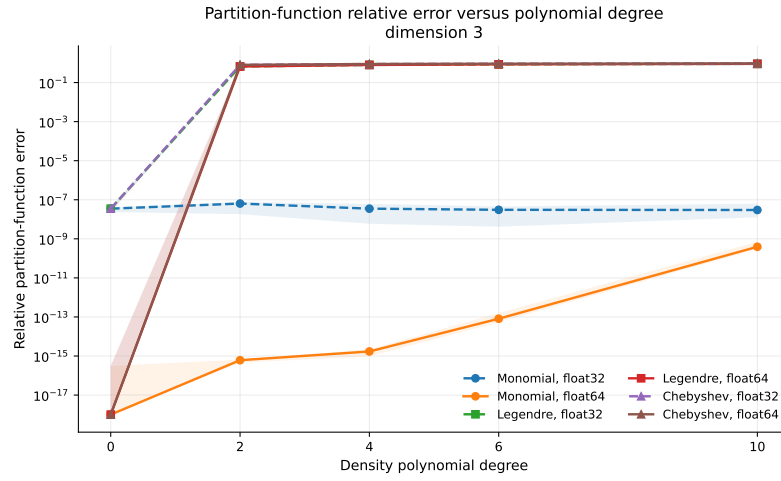


Figure 5.9: Partition-function relative error versus density degree in the representative PAL benchmark.

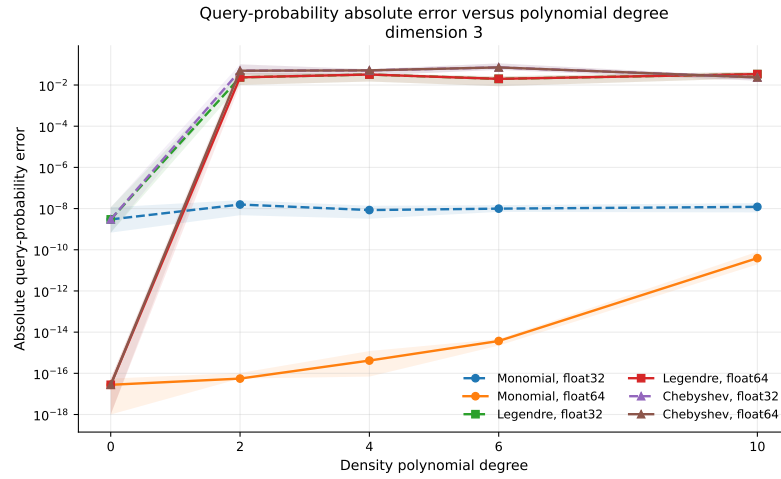


Figure 5.10: Absolute normalised query-probability error versus density degree in the representative PAL benchmark.

introduced during coefficient construction or conversion.

5.7.2 Precision-Sensitive PAL Triangle Case Study

A separate minimal reproducer supplied during project supervision provides a PAL-originated precision stress case. It isolates one triangle from a failing spline cell and evaluates its WMI contribution using PAL’s analytical simplex machinery [1]. The vertices are $(0, a)$, $(1, a)$, and $(1, 1)$, where

$$a = 0.933797909407666.$$

The integrand is a degree-12 monomial product constructed from a squared spline cell and is therefore mathematically non-negative:

$$I_{\Delta} \geq 0. \quad (5.1)$$

The reproducer changes the precision used by the monomial-function evaluation inside the integration path. The reported results are: The `float64` value is not treated as

Table 5.2: Precision-sensitive PAL triangle reproducer. The mathematical integral is non-negative, so the sign is used as a qualitative correctness check.

Path	Reported integral	Sign
Default monomial evaluation (<code>float32</code>)	$-1.7947995740423999 \times 10^{-24}$	Violated
Full monomial evaluation (<code>float64</code>)	$+7.2752680397899133 \times 10^{-24}$	Preserved

arbitrary-precision ground truth; the robust conclusion is only that the expected non-negative sign is preserved in that path and violated by the default lower-precision path. The example is therefore stronger than a modest relative-error increase: finite precision changes a qualitative mathematical property. Such behaviour is consistent with rounding and cancellation in finite-precision arithmetic [22, 3]. The magnitude is extremely small, so the case does not imply that PAL integrals are generally unstable. Nor does it establish that Legendre or Chebyshev representations would remove the failure. A valid cross-basis extension must preserve the same triangle, underlying spline-cell polynomial, and coefficient construction while changing only its representation. The case is therefore used as an independently encountered PAL datapoint and as a targeted regression case for future numerical paths.

5.8 Experimental Synthesis

The experiments do not produce a universal basis ranking. Instead, they expose a sequence of increasingly practical effects. Controlled degree and G_C experiments show that monomial systems can become much more ill-conditioned and lose numerical rank earlier than orthogonal alternatives, yet benign `float64` forward integration can remain at machine-precision scale. Physical scaling, reduced precision, and coefficient recovery make the distinction operational: monomial perturbation sensitivity grows dramatically with scale, `float32` monomial systems lose substantially more rank, and constrained-polytope coefficient noise is amplified by several additional orders of

magnitude. Downstream probability experiments show that arithmetic precision can still dominate basis choice. Coefficient recovery adds another qualification: substantially better recovery conditioning does not guarantee a correspondingly better final integral. The representative PAL benchmark demonstrates the most consequential pipeline effect. Current orthogonal conversion/recovery becomes unstable in extremely ill-conditioned PAL coefficient systems, overwhelming the favourable conditioning seen in controlled experiments. The supervisor-provided triangle case gives a complementary monomial example in which reduced precision changes the sign of a mathematically non-negative integral. The central experimental relationship is therefore

$$\boxed{\text{better representation conditioning} \not\Rightarrow \text{better downstream inference}} \quad (5.2)$$

unless the intervening coefficient-construction, integration, and normalisation stages preserve that numerical advantage.

5.9 Limitations

The conclusions are subject to the following limits:

- The dimension benchmark uses five trials per configuration, limiting resolution of small differences.
- Condition numbers depend on point construction, total-degree truncation, and constraint geometry rather than being universal constants of a polynomial family.
- Coefficient-space perturbations are basis dependent and are not identical function-space perturbations.
- The downstream benchmark uses nested simplices, and the polytope experiments use analytically tractable box-with-obstacle geometries rather than the full range of SMT(LRA) constraints.
- The G_C benchmark is restricted to dimension 2 and two representative regions.
- The orthogonal implementations are experimental and unoptimised, so runtime differences are not intrinsic complexity bounds.
- The representative PAL benchmark exercises the numerical coefficient and integration stages in dimensions 2 and 3, not a complete trained end-to-end neural workload.

- The large PAL errors in the current Legendre and Chebyshev paths are attributed to conversion/recovery and should not be interpreted as intrinsic errors of those polynomial families.

These qualifications bound the claims developed in [Chapter 6](#).

Chapter 6

Discussion

6.1 Overview

The experiments show that polynomial basis choice can materially affect numerical conditioning, but that improved conditioning does not automatically produce more accurate constrained probabilistic inference. Legendre and Chebyshev representations generally retain more numerical rank and reduce coefficient-recovery sensitivity in demanding regimes. However, direct `float64` integration can remain accurate even for poorly conditioned monomial systems, while unstable coefficient construction can erase the advantage of a better-conditioned target basis. The central interpretation is therefore that numerical reliability in PAL is a *pipeline property*. Representation, coordinate scaling, coefficient construction, arithmetic precision, constrained integration, and normalisation must be considered jointly.

6.2 RQ1: How Does Basis Choice Affect Numerical Conditioning?

The degree experiments establish a clear basis effect. At high degree, sampled monomial systems become substantially more ill-conditioned than the corresponding Legendre and Chebyshev systems, consistent with the classical sensitivity of Vandermonde-type constructions [2, 3, 8, 4]. The higher-dimensional results qualify this ordering. Orthogonal advantages are strongest in lower-dimensional, higher-degree configurations and narrow as the multivariate basis grows. By dimension 5 at degree 5, all three sampled systems have condition numbers of the same order. This reflects the fact that

the measured system depends not only on the one-dimensional polynomial family, but also on sample placement, total-degree truncation, coordinate mapping, and dimension. The constraint-induced G_C benchmark provides stronger PAL-relevant evidence because it replaces sampled evaluation with the inner product imposed by the feasible region. On the unit simplex, monomial G_C loses numerical rank earlier than the orthogonal alternatives. On the box-with-obstacle region, the difference is much larger: at degree 10, the monomial matrix is numerically singular while Legendre and Chebyshev remain full rank. Thus:

Orthogonal polynomial representations can substantially improve conditioning and numerical-rank retention, but the magnitude and ordering of that advantage depend on the complete discretised system and constrained geometry.

6.3 RQ2: When Does Conditioning Become Practically Important?

Poor conditioning does not imply immediate forward failure. In the controlled simplex experiments, condition numbers increase by many orders of magnitude while direct `float64` integration remains close to machine precision. This is consistent with the standard interpretation of condition numbers as measures of potential perturbation amplification rather than guarantees that a specific computation realises the worst case [3, 11]. Three regimes expose the latent sensitivity. **Physical scale.** Large coordinate magnitude produces the strongest controlled perturbation effect. High-degree monomial powers span increasingly different magnitudes, whereas the orthogonal representations continue to operate through explicit canonical coordinates. This supports stable canonicalisation when compatible with the probabilistic semantics. **Reduced precision.** At degree 8 in three dimensions, the `float32` monomial system retains only about 35% of its numerical rank, compared with approximately 82%–85% for Legendre and Chebyshev. Numerical-rank loss is more consequential than a large finite condition number because unresolved basis directions cannot be recovered reliably. The current implementation also shows little runtime benefit from `float32`, so reduced precision is not justified solely as a speed optimisation for conditioning-sensitive stages. **Coefficient recovery.** In the constrained-polytope benchmark, monomial coefficient-noise amplification reaches approximately 10^6 at the largest tested degree, compared with approximately 10^1 for the orthogonal alternatives. Recovery directly exercises

the inverse problem associated with V , turning potential conditioning sensitivity into realised coefficient error. The answer to RQ2 is therefore:

Conditioning differences become practically important when perturbations interact with sensitive basis directions. Large physical scale, reduced precision, and coefficient recovery expose these effects much more clearly than benign double-precision forward integration.

6.4 RQ3: Do Orthogonal Bases Improve Constrained Integration?

The answer depends on which stage of constrained integration is considered. When accurate coefficients are already available, the experiments show no universal orthogonal-basis forward-error advantage: the analytical monomial path can remain extremely accurate even when its associated sampled recovery system is badly conditioned. When coefficients must first be reconstructed, the situation changes. Orthogonal representations retain more numerical rank and amplify sampled-value noise by several orders of magnitude less than monomials at high degree. Yet the recovered-integral errors remain roughly 10^{-6} – 10^{-5} across all three bases. This can be represented as

$$\text{samples} \rightarrow \text{coefficients} \rightarrow \text{constrained integral.} \quad (6.1)$$

Each transformation contributes numerical error. Improving recovery conditioning therefore need not improve the final integral if a later stage dominates. Hence:

Orthogonal bases provide clear high-degree stability advantages during coefficient recovery, but lower recovery sensitivity does not automatically imply lower constrained-integration error.

6.5 RQ4: Do Numerical Effects Reach Probabilistic Inference?

The controlled downstream benchmark shows that integration error propagates into partition functions and normalised queries, but arithmetic precision is the dominant observed effect. Within each arithmetic type, median probability errors are comparatively similar across the three bases. The representative PAL benchmark exposes a different failure mode. The current Legendre and Chebyshev paths develop approximately

order-one partition-function error and order- 10^{-2} query error from density degree 2, while the direct monomial baseline remains substantially more accurate. This does not contradict the favourable orthogonal conditioning observed in controlled experiments. Those experiments establish that the target representation can be better conditioned; they do not establish that every procedure for constructing its coefficients is stable. If conversion or recovery no longer reproduces the intended mathematical polynomial, subsequent integration may be accurate for the wrong numerical function. The relevant transformation is therefore

$$\text{model output} \rightarrow \text{scaled representation} \rightarrow \text{stable coefficient construction} \rightarrow \text{constrained integral} \rightarrow \text{normalised probabilities} \quad (6.2)$$

The supervisor-provided triangle reproducer adds a complementary precision-specific example. There, the default lower-precision monomial path returns a negative value for a mathematically non-negative integral, while the full `float64` path preserves the expected sign. This shows that arithmetic precision can become a correctness issue, not merely an accuracy metric, in sufficiently cancellation-sensitive PAL computations [22, 3]. Thus:

Polynomial numerical errors can propagate into PAL partition functions and normalised probabilities. Arithmetic precision dominates the controlled downstream benchmark, while unstable coefficient construction can dominate the representative PAL pipeline.

6.6 Conditioning Is Necessary but Not Sufficient

The original hypothesis was that replacing monomials with better-conditioned representations could improve the reliability of PAL’s constrained integration. The experiments strongly support the conditioning component but qualify the downstream claim. A useful conceptual decomposition is

$$E_{\text{total}} = E_{\text{construction}} + E_{\text{representation}} + E_{\text{recovery}} + E_{\text{integration}} + E_{\text{normalisation}}, \quad (6.3)$$

where this is a conceptual separation of error sources rather than a literal floating-point identity. Basis choice primarily targets representation conditioning and part of coefficient recovery. It cannot compensate automatically for a larger error in another stage. The controlled simplex benchmark illustrates a case where conditioning changes greatly but direct integration remains accurate; the polytope benchmark shows improved recovery conditioning without a comparable reduction in final integral error; and

the representative PAL benchmark shows coefficient construction becoming unstable enough to overwhelm the target basis. The central technical conclusion is therefore

$$\text{numerically stable PAL} \neq \text{better basis alone}, \quad (6.4)$$

and instead requires the complete numerical chain

$$\begin{aligned} &\text{stable representation} + \text{stable coefficient construction} \\ &+ \text{stable constrained integration} + \text{stable normalisation}. \end{aligned} \quad (6.5)$$

6.7 Implications for PAL

The results suggest four practical design principles. **Explicit canonical scaling.** High-degree sensitivity depends strongly on physical coordinate magnitude. Canonical mappings should therefore be explicit and stable rather than changing implicitly with the feasible constraint [8, 9, 10]. **Rank-aware coefficient construction.** Reduced-precision and recovery experiments show that nominal basis directions can become numerically unresolved. Rank-revealing QR, truncated singular-value methods, or regularised least squares are therefore preferable to assuming exact full rank [11, 3]. **Avoid unnecessary conversion.** The representative PAL failure suggests that passing a high-degree polynomial through an ill-conditioned sampled conversion system can eliminate the benefit of the target basis. Where possible, coefficients should be produced directly in the desired representation rather than repeatedly reconstructed through sampled values or monomial coefficients. **Use precision selectively.** The experiments do not support indiscriminate use of `float32`. Coefficient recovery, conversion, accumulation, and partition-function evaluation are plausible candidates for `float64` or other higher-stability arithmetic even when surrounding neural operations use reduced precision. The PAL triangle sign failure provides a concrete regression test for such mixed-precision designs. An adaptive strategy is also plausible: low-degree, well-scaled problems can retain the inexpensive monomial path, while rank or conditioning diagnostics can trigger alternative representations or solvers in difficult regimes.

6.8 Relation to Prior Work and Future Directions

Weighted Model Integration combines logical constraints with continuous integration [17, 18], while PAL embeds this principle in a differentiable neuro-symbolic probabilistic layer [1]. Numerical analysis independently documents the sensitivity of high-degree

monomial and Vandermonde-type representations [2, 3], with orthogonal polynomials, barycentric formulations, and splines providing alternative approximation tools [20, 4, 13]. The present results connect these areas by showing that polynomial representation can determine how much numerical information survives inside constrained probabilistic inference. They also demonstrate that selecting a better-conditioned family is insufficient unless coefficient construction and later numerical stages preserve its properties. The most immediate future work is therefore to redesign the unstable coefficient-construction path using rank-revealing decompositions, regularisation, improved scaling, recurrence-based evaluation, or direct prediction of orthogonal coefficients. The G_C benchmark should be extended to higher-dimensional and more general SMT(LRA) regions. Local representations such as splines are also promising because they may avoid global high-degree sensitivity, although they introduce piecewise structure and knot-selection decisions [13]. For constrained regions without analytical moments, independent high-accuracy numerical integration can provide additional consistency checks [21]. Ultimately, any redesigned path should be evaluated in a complete trained PAL workload to determine whether improved numerical reliability is achieved without unacceptable computational overhead.

6.9 Summary

Legendre and Chebyshev representations can materially improve conditioning, numerical-rank retention, and coefficient-recovery robustness in demanding regimes. These advantages do not guarantee lower forward or downstream error: benign monomial integration can remain accurate, later stages can mask recovery improvements, and unstable coefficient construction can erase the benefits of the target basis entirely. Reliable basis replacement in PAL therefore requires an end-to-end numerical design in which representation, scaling, coefficient construction, constrained integration, arithmetic precision, and normalisation are treated jointly.

Chapter 7

Conclusion

7.1 Summary and Answer to the Research Question

This dissertation investigated whether replacing PAL’s monomial polynomial representation with Legendre or Chebyshev bases can improve the numerical reliability of constrained probabilistic inference [1]. Equivalent mathematical polynomials were compared while varying degree, dimension, physical scale, arithmetic precision, coefficient perturbations, constraint geometry, and coefficient recovery. The evaluation then extended to partition functions, normalised queries, changing constraints, and a representative PAL numerical pipeline. The results establish that basis choice can substantially affect numerical conditioning. Monomial systems frequently become more ill-conditioned at high degree and lose substantially more numerical rank in `float32`. The constraint-induced G_C experiment further shows that these differences can persist under the feasible-region integration geometry rather than being confined to sampled interpolation matrices. However, improved conditioning does not imply automatically improved forward accuracy. Controlled `float64` simplex integration remains close to machine precision across all three representations despite large condition number differences. Conditioning becomes practically important under large physical scale, reduced precision, and coefficient recovery, where monomials show stronger perturbation amplification and rank loss. The PAL-originated triangle stress case adds a qualitative precision result: the default lower-precision path returns a negative value for a mathematically non-negative spline-cell integral, while the full-`float64` route preserves the expected sign. This demonstrates that finite precision can become a correctness issue in sufficiently cancellation-sensitive computations. The central research question asked whether better polynomial representations translate into more accurate PAL-relevant

inference. The experiments support a qualified answer: Legendre and Chebyshev can materially improve conditioning, rank retention, and coefficient-recovery robustness, but these advantages survive only when coefficient construction and subsequent numerical stages preserve the intended representation. The main result is therefore

$$\boxed{\text{better-conditioned basis} \not\Rightarrow \text{more accurate PAL inference}} \quad (7.1)$$

without a correspondingly stable coefficient-construction and constrained-integration pipeline.

7.2 Implications and Future Work

A more robust PAL implementation should combine explicit canonical scaling, rank-aware coefficient construction, stable or direct formation of coefficients in the selected basis, appropriate precision for conditioning-sensitive stages, and constrained integration and accumulation methods that preserve the representation. The existing monomial path remains attractive for low-degree, well-scaled problems where it is accurate and inexpensive. Alternative representations are most relevant when degree, coordinate scale, finite precision, or numerical recovery exposes monomial sensitivity. This suggests adaptive basis or solver selection rather than unconditional replacement. The most immediate extension is to redesign the unstable coefficient path identified by the representative PAL benchmark. Rank-revealing factorizations, regularisation, improved scaling, recurrence-based evaluation, and direct prediction of orthogonal coefficients should be tested on the same difficult systems [11, 3, 10]. The G_C analysis should also be extended to higher-dimensional and more general SMT(LRA) geometries. Local representations such as splines remain another possible direction because their support may avoid some global high-degree sensitivity, although they introduce piecewise structure and knot-selection choices [13]. Finally, any redesigned numerical path should be tested in a complete trained PAL workload.

7.3 Final Conclusion

PAL's monomial representation can become numerically fragile in high-degree, large-scale, reduced-precision, and coefficient-recovery regimes. Legendre and Chebyshev representations often mitigate these effects and can preserve substantially more numerical information. Their benefit nevertheless depends on the complete computation. A

better-conditioned representation provides limited value if its coefficients are produced through an unstable transformation or if subsequent operations destroy the improvement. Reliable constrained probabilistic inference therefore requires an end-to-end stable numerical path:

representation $\rightarrow$ coefficient construction $\rightarrow$ constrained integration $\rightarrow$ normalisation.
--

(7.2)

Orthogonal polynomial bases remain promising components of such a system, but their numerical properties must be preserved throughout the PAL pipeline.

Bibliography

- [1] Leander Kurscheidt, Paolo Morettin, Roberto Sebastiani, Andrea Passerini, and Antonio Vergari. A probabilistic neuro-symbolic layer for algebraic constraint satisfaction. In Silvia Chiappa and Sara Magliacane, editors, *Proceedings of the Forty-first Conference on Uncertainty in Artificial Intelligence*, volume 286 of *Proceedings of Machine Learning Research*, pages 2431–2471. PMLR, 2025.
- [2] Walter Gautschi. On inverses of vandermonde and confluent vandermonde matrices. *Numerische Mathematik*, 4:117–123, 1962.
- [3] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2 edition, 2002.
- [4] Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2013.
- [5] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.
- [6] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- [7] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [8] Gábor Szegő. *Orthogonal Polynomials*. American Mathematical Society, 4 edition, 1975.
- [9] Theodore J. Rivlin. *Chebyshev Polynomials*. Wiley, 2 edition, 1990.
- [10] John P. Boyd. *Chebyshev and Fourier Spectral Methods*. Dover Publications, 2 edition, 2001.
- [11] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4 edition, 2013.

- [12] Alfio Quarteroni, Riccardo Sacco, and Fausto Saleri. *Numerical Mathematics*, volume 37 of *Texts in Applied Mathematics*. Springer, 2 edition, 2007.
- [13] Carl de Boor. *A Practical Guide to Splines*. Springer, 2001.
- [14] Clark Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. Satisfiability modulo theories. In *Handbook of Satisfiability*, pages 825–885. IOS Press, 2009.
- [15] Günter M. Ziegler. *Lectures on Polytopes*. Springer, 1995.
- [16] Mark Chavira and Adnan Darwiche. On probabilistic inference by weighted model counting. *Artificial Intelligence*, 172(6–7):772–799, 2008.
- [17] Vaishak Belle, Andrea Passerini, and Guy Van den Broeck. Probabilistic inference in hybrid domains by weighted model integration. In *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence*, pages 2770–2776. IJCAI Inc., 2015.
- [18] Paolo Morettin, Andrea Passerini, and Roberto Sebastiani. Efficient weighted model integration via SMT-based predicate abstraction. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 720–728, 2017.
- [19] John C. Mason and David C. Handscomb. *Chebyshev Polynomials*. Chapman and Hall/CRC, 2002.
- [20] Jean-Paul Berrut and Lloyd N. Trefethen. Barycentric lagrange interpolation. *SIAM Review*, 46(3):501–517, 2004.
- [21] Philip J. Davis and Philip Rabinowitz. *Methods of Numerical Integration*. Dover Publications, 2007.
- [22] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

before onto

Appendix A

Validation and Verification

This appendix records supporting validation checks. The principal checks required to interpret the results are discussed in the main text; this record is retained for reproducibility.

Table A.1: Validation checks used throughout the experimental implementation.

Check	Purpose
Basis equivalence	Confirms that monomial, Legendre, and Chebyshev coefficients reproduce the same mathematical polynomial at independent points.
Canonical mapping	Checks physical-domain endpoints against the canonical interval $[-1, 1]$.
Multi-index consistency	Confirms common ordering across coefficient vectors, basis matrices, conversion matrices, and integration weights.
One-dimensional simplex agreement	Checks that the general simplex routine reproduces ordinary integration over $\Delta_1 = [0, 1]$.
Analytical simplex moments	Checks Equation (2.21) against independently verifiable low-dimensional cases.
Higher-dimensional basis counts	Checks the generated basis size against $M(n, d) = \binom{n+d}{d}$.
Reduced-precision execution	Checks finite float32/float64 outputs and numerical-rank diagnostics.

Continued on the next page.

Table A.1 continued from the previous page.

Check	Purpose
Generated-density positivity	Confirms that $q^2 + 0.1$ remains strictly positive at validation points.
Partition/query consistency	Checks finite normalised probabilities and containment of query regions.
Coefficient recovery	Checks finite least-squares residuals and noiseless recovery in sufficiently well-conditioned cases.
Static constrained domains	Checks outer-box and obstacle moment calculations.
Dynamic constraint schedules	Checks that all changing obstacles remain inside the fixed ambient domain.
Polytope configuration consistency	Confirms that the completed benchmark and regenerated degree plots use only dimensions 2, 3 and density degrees 0, 2, 4, 6, 10; no dimension-4 or density-degree-12 polytope row is present.
Deterministic seeds	Confirms reproducibility of source polynomials, samples, and perturbation directions.
Structured results	Checks required output fields and finite core metrics before plotting.
PAL baseline regression	Confirms that the original monomial PAL path remains operational while the experimental alternatives are evaluated separately.
Representative PAL diagnostics	Records representation-equivalence residuals, recovery residuals, rank, scaling parameters, and the sampled recovery matrix.
Constraint-induced G_C validation	Checks symmetry, low-degree analytical entries, positive semidefiniteness up to floating-point tolerance, numerical rank, and treatment of singular configurations.

A.1 Representative Smoke Tests

Listing A.1: Representative compilation and smoke-test commands.

```
python -m compileall analysis src tests
python -m analysis.simplex.run_dimension_benchmark \
--dimensions 1 2 \
--maximum-degree 3 \
--trials 2
```

A.2 Constraint-Induced Gram Validation

For the constant basis,

$$G_C = \left[\text{vol}(C) \right], \quad \kappa_2(G_C) = 1. \quad (\text{A.1})$$

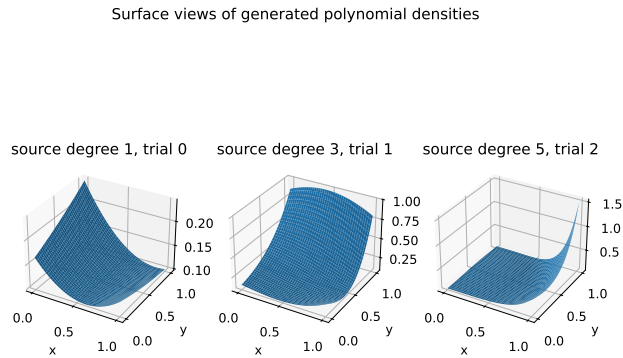
The recorded degree-0 values are 0.5 for the unit simplex and 0.84 for the unit square after removing the centred obstacle, matching the corresponding areas. At higher degree, numerical singularity is retained explicitly rather than replacing an unresolved singular direction with a finite condition number.

Appendix B

Generated Polynomial Examples

This appendix retains visual checks of the deterministic positive-density generator and cross-basis equivalence. These figures are diagnostic rather than additional evidence of basis superiority.

B.1 Two-Dimensional Surface Views



These surfaces show the same densities as the heatmap figure.

Figure B.1: Surface views of representative two-dimensional generated polynomial densities.

B.2 Cross-Basis Equivalence

The discrepancy in the representative example remains approximately at double-precision rounding scale.

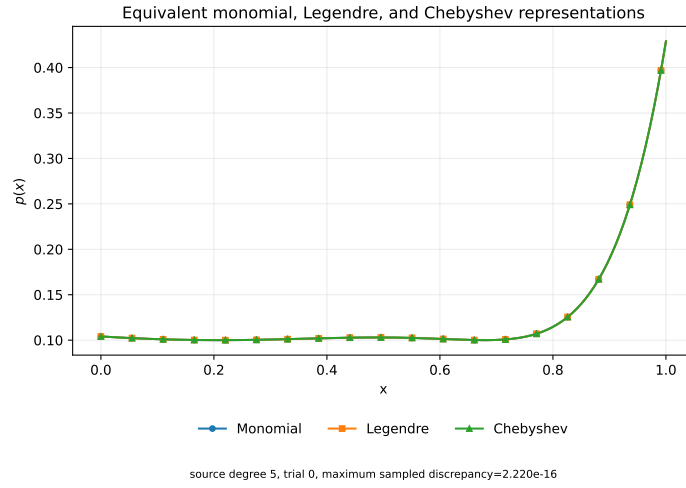


Figure B.2: Representative equivalence check for monomial, Legendre, and Chebyshev representations of the same polynomial.

B.3 Representative Constrained Region

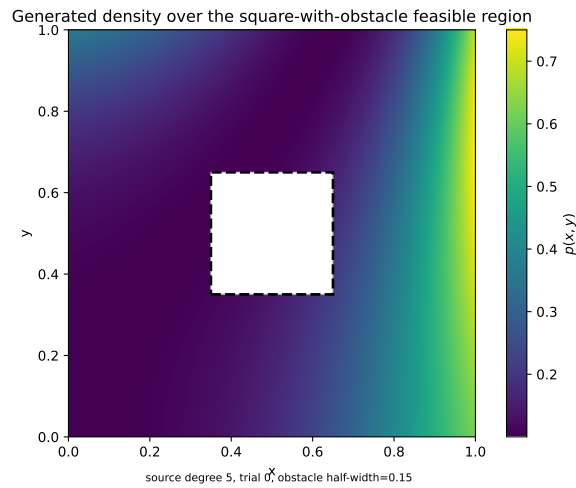


Figure B.3: Representative generated density and feasible subset within a fixed ambient polynomial domain.

B.4 Supplementary Benchmark Figures

The following plots retain supporting evidence removed from the counted main body during page-limit tightening.

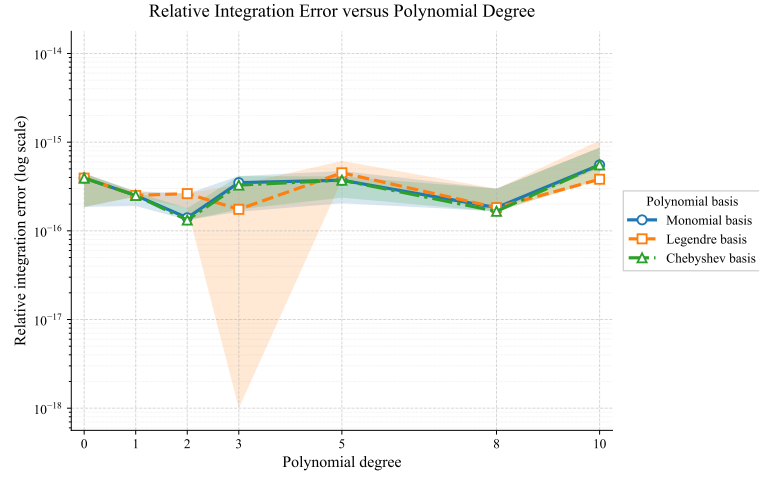


Figure B.4: Supplementary direct integration error versus degree.

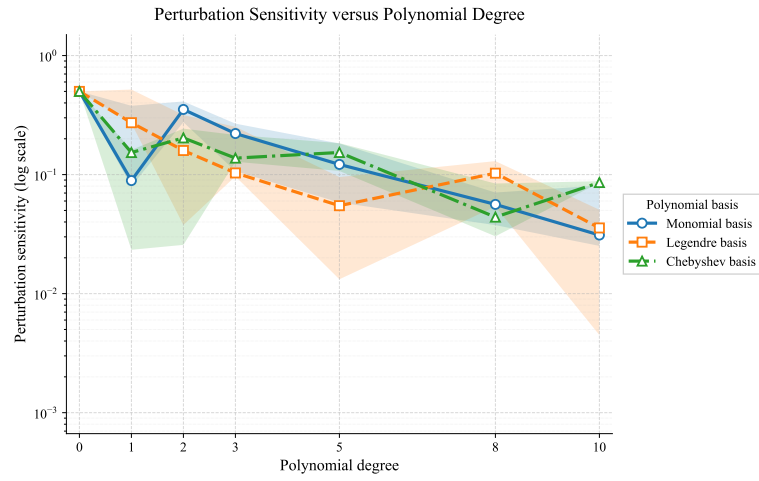
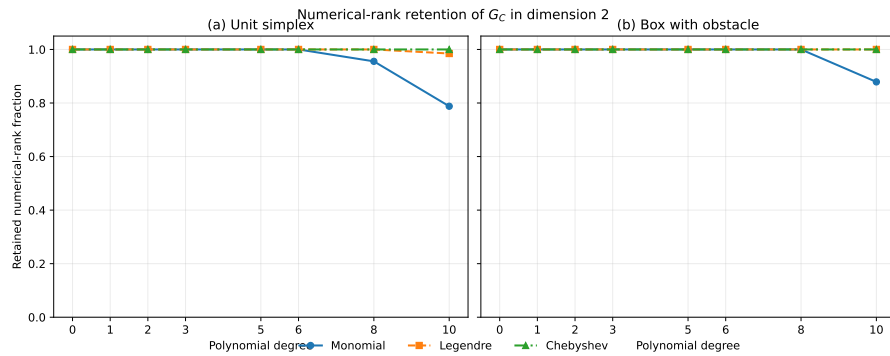


Figure B.5: Supplementary coefficient-perturbation sensitivity versus degree.

Figure B.6: Supplementary retained numerical-rank fraction of G_C .

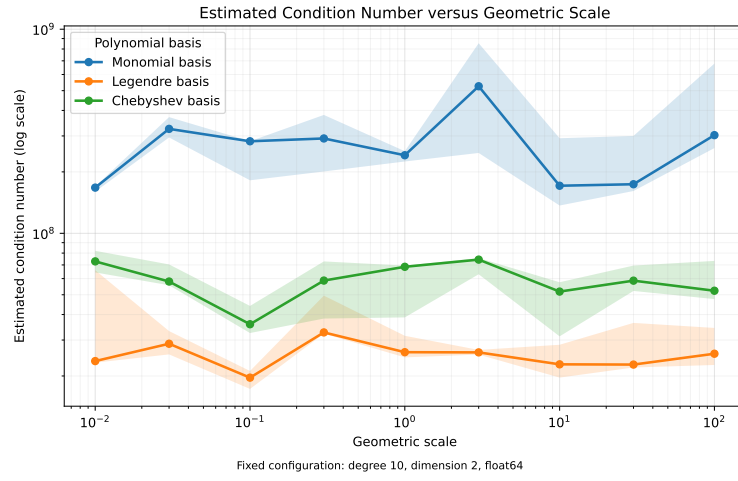


Figure B.7: Supplementary sampled-system conditioning versus physical scale.

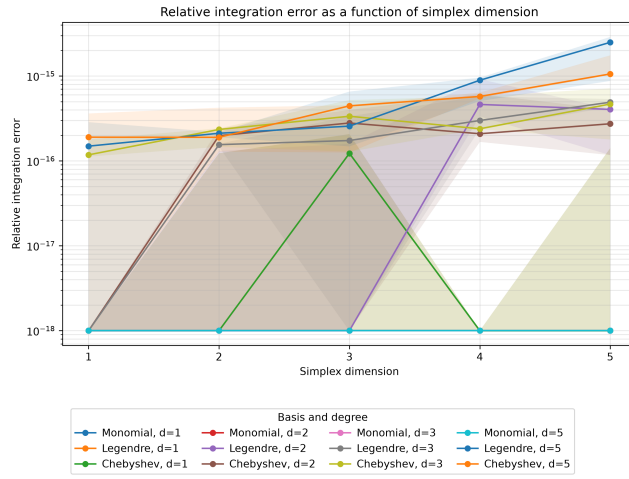


Figure B.8: Supplementary direct integration error versus dimension.

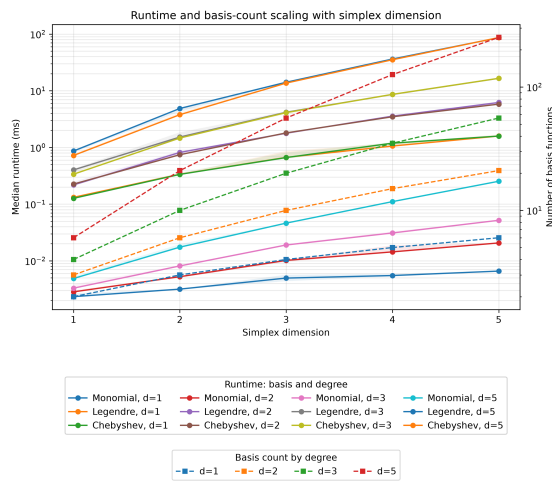


Figure B.9: Supplementary runtime and total-degree basis count versus dimension.

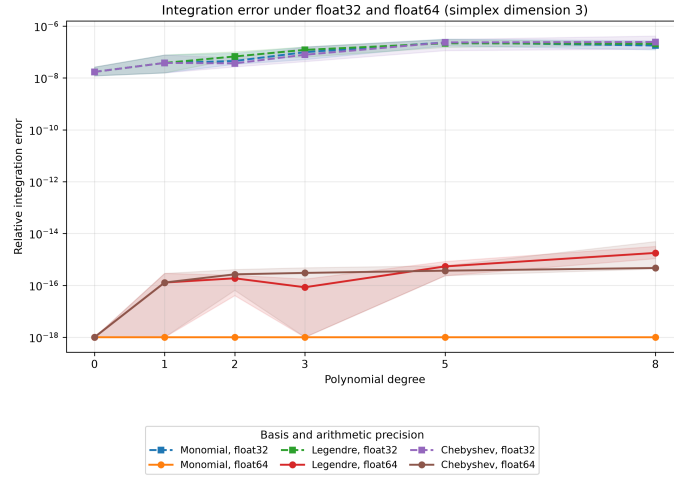


Figure B.10: Supplementary integration error under reduced precision.

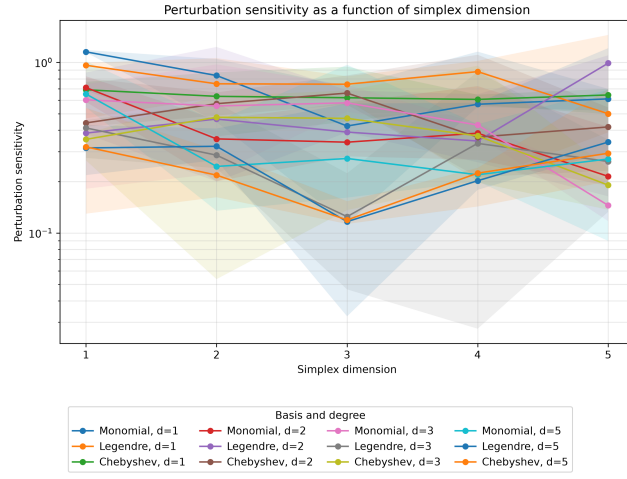


Figure B.11: Supplementary perturbation sensitivity versus dimension.

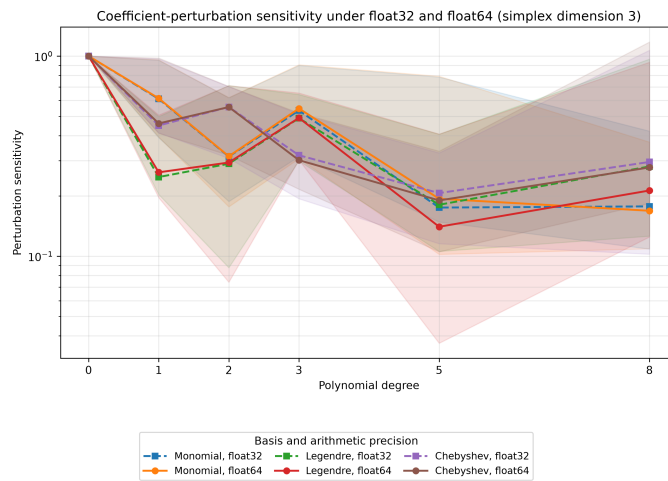


Figure B.12: Supplementary perturbation sensitivity under reduced precision.

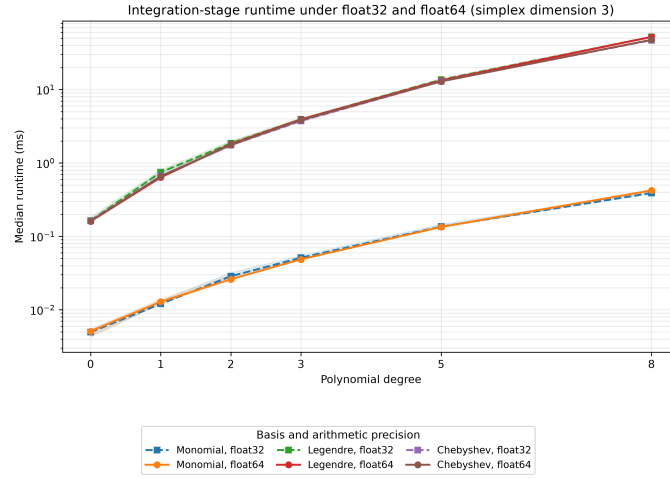


Figure B.13: Supplementary runtime comparison by arithmetic precision.

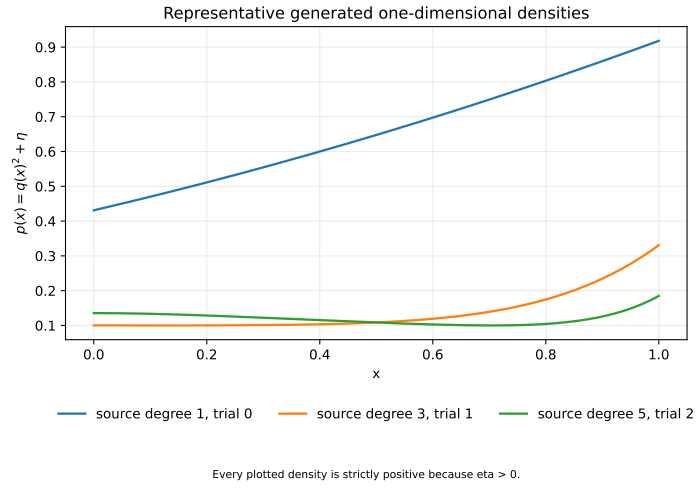


Figure B.14: Representative one-dimensional generated positive densities.

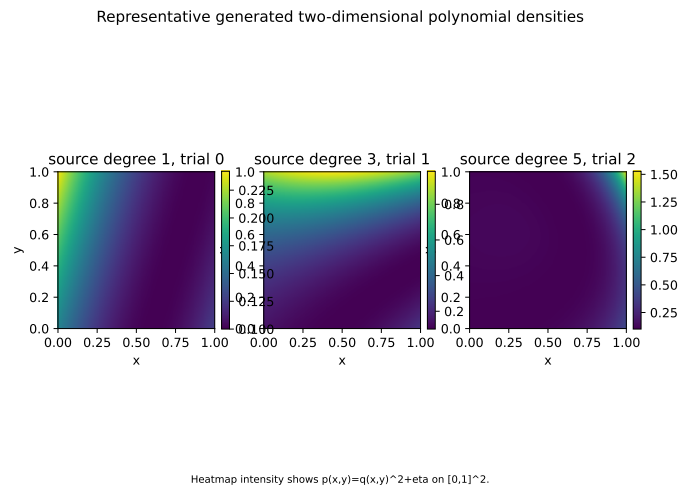


Figure B.15: Representative two-dimensional generated density heatmaps.

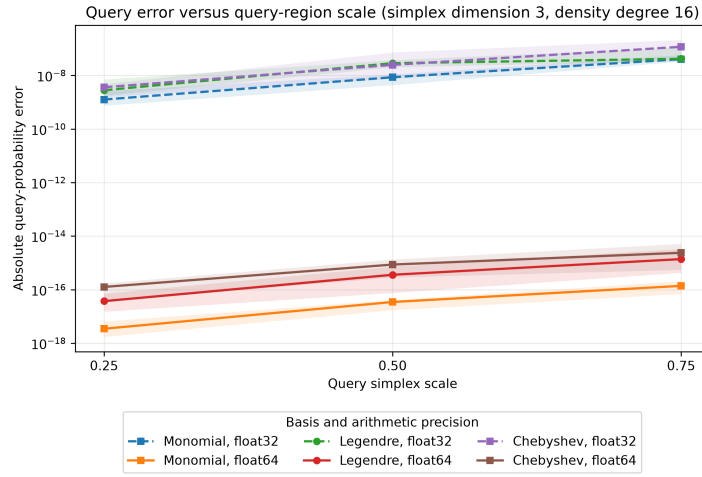


Figure B.16: Supplementary query error versus query-region scale.

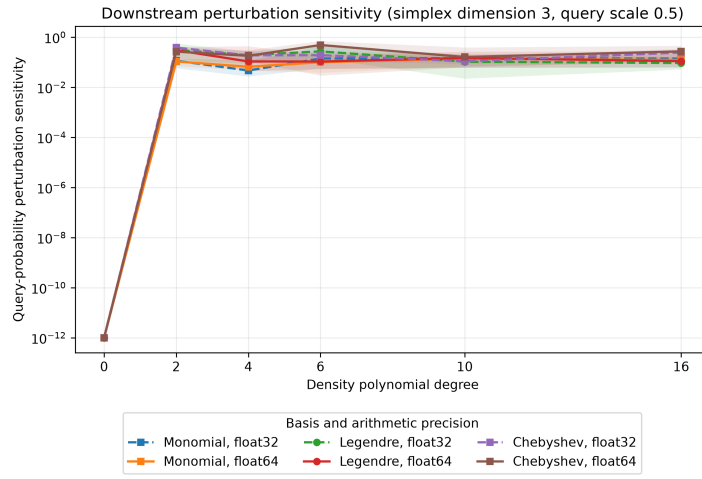


Figure B.17: Supplementary query-probability perturbation sensitivity versus degree.

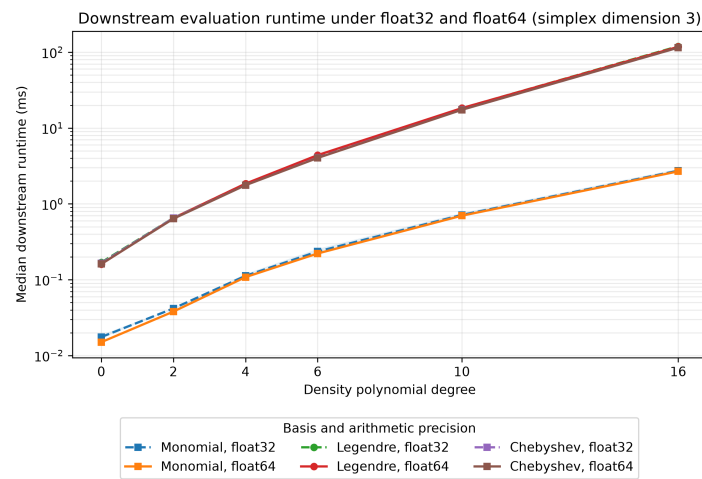


Figure B.18: Supplementary downstream runtime versus density degree.

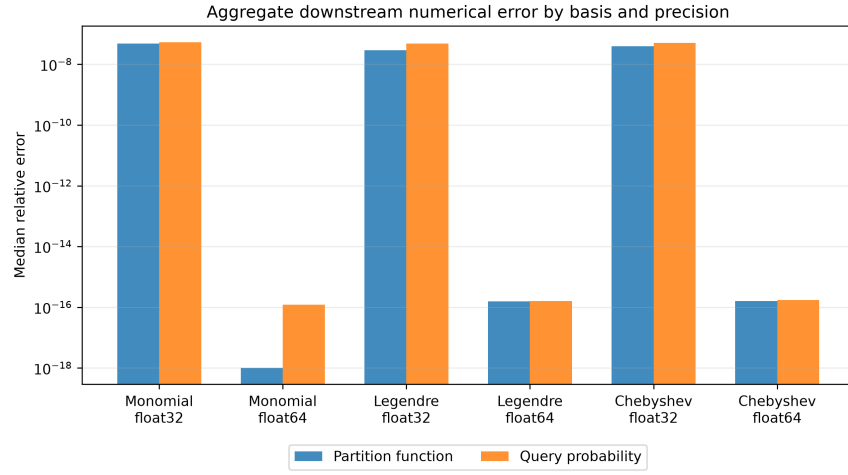


Figure B.19: Supplementary aggregate downstream-error summary.

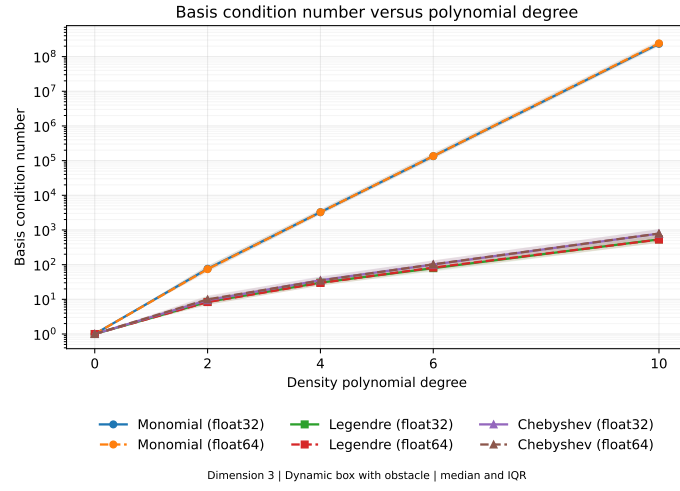


Figure B.20: Supplementary constrained-polytope condition number versus degree.

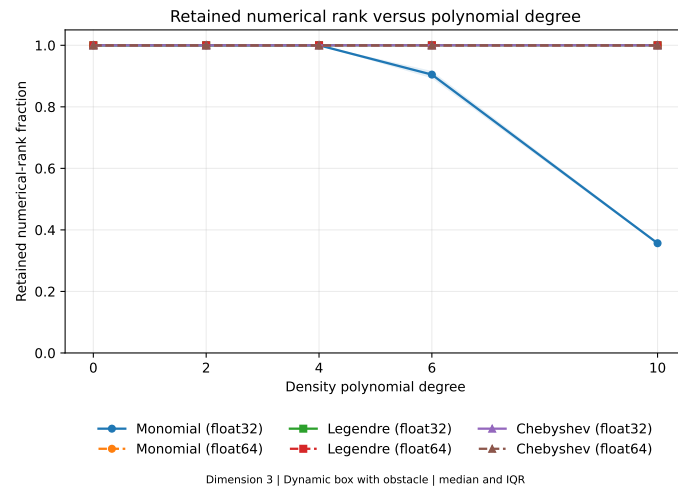


Figure B.21: Supplementary constrained-polytope retained numerical rank.

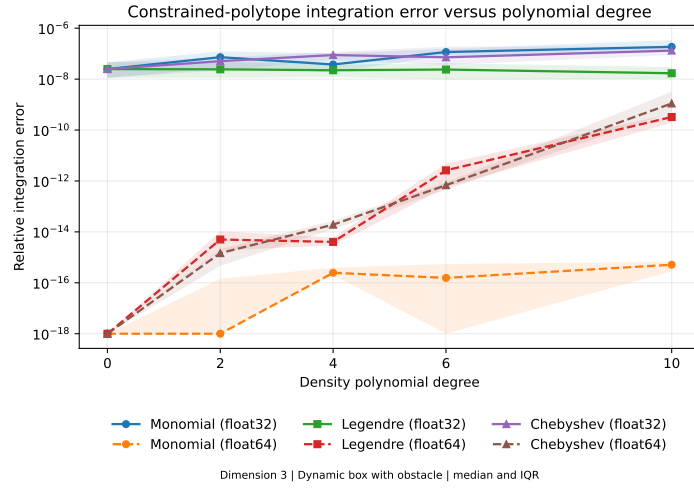


Figure B.22: Supplementary direct constrained-polytope integration error.

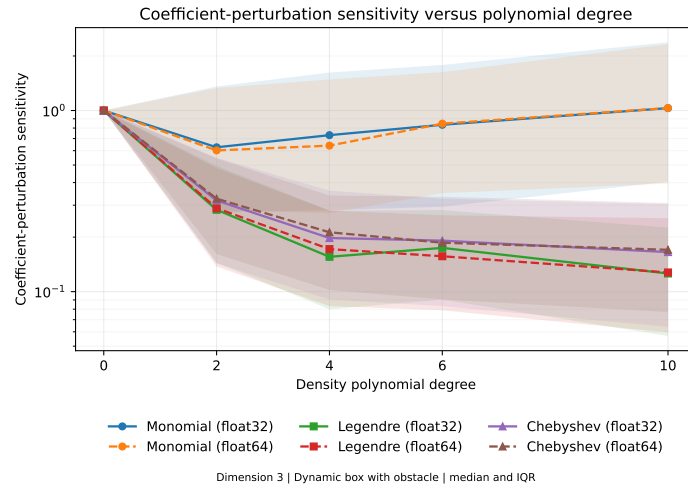


Figure B.23: Supplementary constrained-polytope perturbation sensitivity.

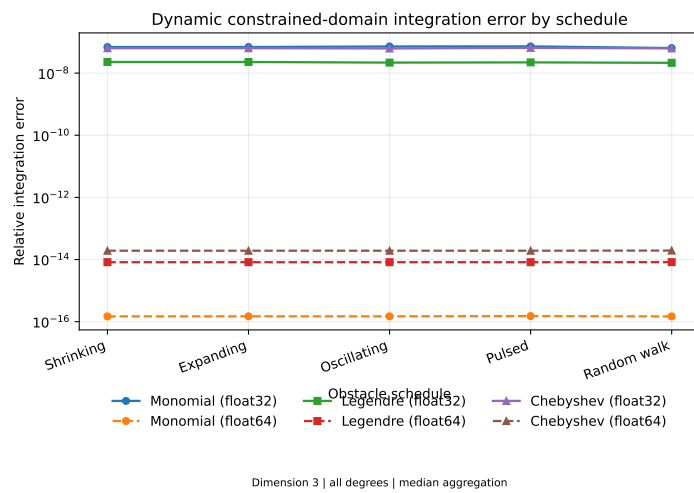


Figure B.24: Supplementary error comparison across changing-constraint schedules.

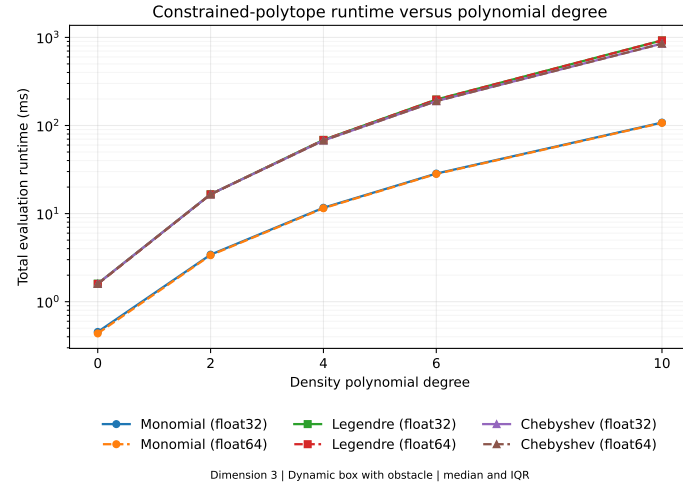


Figure B.25: Supplementary constrained-polytope runtime versus degree.

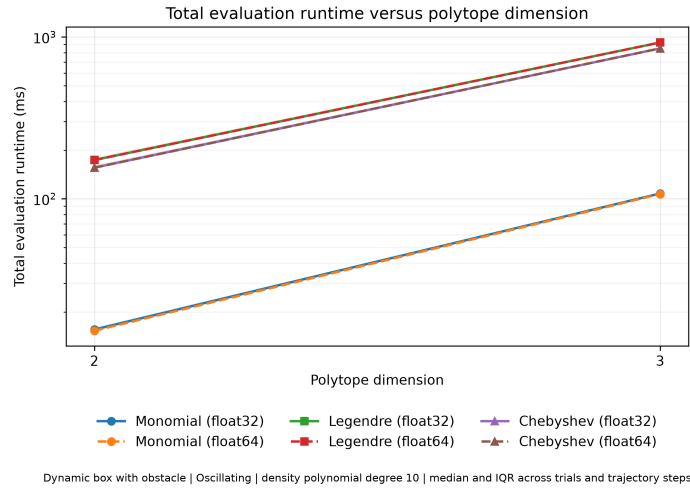


Figure B.26: Supplementary constrained-polytope runtime versus dimension.

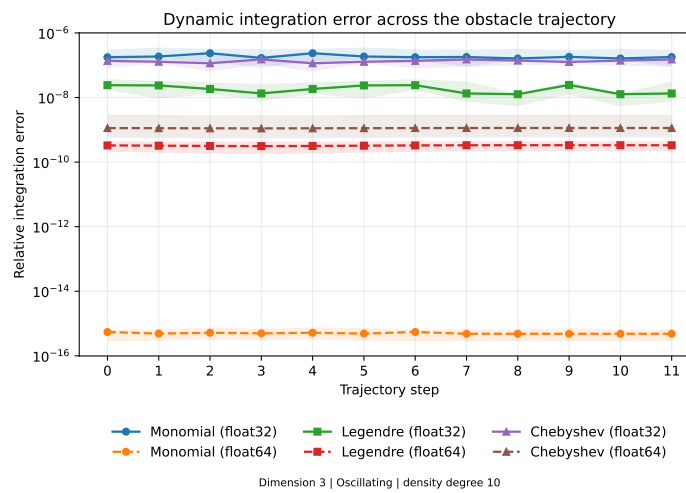


Figure B.27: Supplementary error along changing-constraint trajectories.

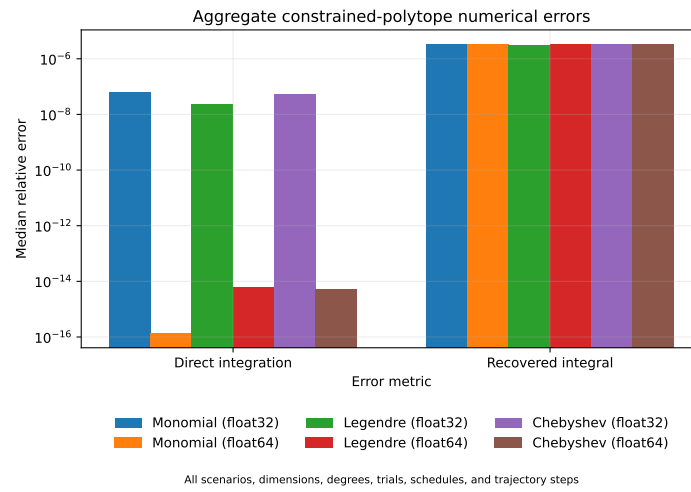


Figure B.28: Supplementary aggregate constrained-polytope error summary.

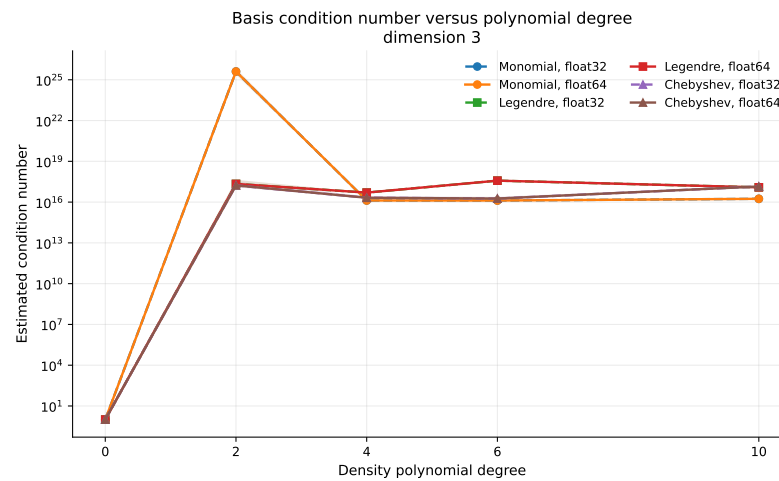


Figure B.29: Supplementary sampled conditioning in the representative PAL benchmark.

Appendix C

Selected Numerical Tables

This appendix records selected numerical values underlying the main figures. Complete row-level results remain available in the repository output files.

C.1 Degree Benchmark

Table C.1: Selected median sampled-system condition numbers and integration errors in the two-dimensional degree benchmark.

	Monomial	Legendre	Chebyshev
<i>Condition number</i>			
Degree 5	1.08×10^4	3.02×10^3	6.89×10^3
Degree 10	3.24×10^8	3.33×10^7	5.25×10^7
<i>Relative integration error</i>			
Degree 5	3.73×10^{-16}	4.51×10^{-16}	3.73×10^{-16}
Degree 10	5.54×10^{-16}	3.82×10^{-16}	5.54×10^{-16}

C.2 Physical-Scale Benchmark

Table C.2: Selected median sampled-system condition numbers versus physical scale.

Scale	Monomial	Legendre	Chebyshev
0.01	1.67×10^8	2.37×10^7	7.29×10^7
1	2.41×10^8	2.62×10^7	6.85×10^7
3	5.25×10^8	2.61×10^7	7.43×10^7
100	3.03×10^8	2.57×10^7	5.23×10^7

C.3 Higher-Dimensional Benchmark

Table C.3: Median degree-5 sampled condition numbers as dimension increases.

Dimension	Monomial	Legendre	Chebyshev
1	2.3276×10^4	4.4490×10^1	5.5454×10^1
2	9.9489×10^4	5.3948×10^3	6.8777×10^3
3	5.8226×10^5	1.0832×10^5	1.4411×10^5
4	2.6439×10^6	1.2115×10^6	1.7108×10^6
5	5.4988×10^6	5.8847×10^6	7.1769×10^6

Table C.4: Median degree-5 relative integration errors as dimension increases.

Dimension	Monomial	Legendre	Chebyshev
1	0	1.4927×10^{-16}	1.9038×10^{-16}
2	0	2.1168×10^{-16}	1.8964×10^{-16}
3	0	2.5672×10^{-16}	4.4524×10^{-16}
4	0	8.9225×10^{-16}	5.7456×10^{-16}
5	0	2.4933×10^{-15}	1.0578×10^{-15}

C.4 Reduced Precision

Table C.5: Aggregate relative integration error by arithmetic precision.

Precision	Median	Q1	Q3	Maximum
float32	5.9200×10^{-8}	2.5374×10^{-8}	1.0918×10^{-7}	9.2102×10^{-7}
float64	0	0	2.1212×10^{-16}	9.5447×10^{-15}

Table C.6: Median retained numerical-rank fraction at degree 8.

n	float32			float64		
	Mono.	Leg.	Cheb.	Mono.	Leg.	Cheb.
1	0.889	1.000	1.000	1.000	1.000	1.000
2	0.644	0.956	0.956	1.000	1.000	1.000
3	0.352	0.824	0.836	1.000	1.000	1.000

C.5 Downstream Probability Benchmark

Table C.7: Aggregate median downstream metrics by basis and precision.

Basis	Precision	Partition error	Query error	Sensitivity	Runtime (ms)
Monomial	float32	4.8814×10^{-8}	9.9181×10^{-9}	0.0997	0.0473
Monomial	float64	0	3.4694×10^{-18}	0.1051	0.0402
Legendre	float32	2.9125×10^{-8}	9.8296×10^{-9}	0.1079	2.3684
Legendre	float64	1.5644×10^{-16}	2.7756×10^{-17}	0.1113	2.3355
Chebyshev	float32	3.9718×10^{-8}	1.0420×10^{-8}	0.1411	2.1521
Chebyshev	float64	1.6023×10^{-16}	2.7756×10^{-17}	0.1384	2.1700

C.6 Constraint-Induced Conditioning

Table C.8: Selected G_C condition numbers and retained numerical-rank fractions.

Domain	Basis	Degree	$\kappa_2(G_C)$	Rank
Unit simplex	Monomial	6	1.733×10^{11}	1.000
Unit simplex	Legendre	6	2.072×10^8	1.000
Unit simplex	Chebyshev	6	3.811×10^8	1.000
Unit simplex	Monomial	8	∞	0.956
Unit simplex	Legendre	8	2.065×10^{11}	1.000
Unit simplex	Chebyshev	8	3.964×10^{11}	1.000
Unit simplex	Monomial	10	∞	0.788
Unit simplex	Legendre	10	∞	0.985
Unit simplex	Chebyshev	10	4.658×10^{13}	1.000
Box with obstacle	Monomial	8	1.515×10^{12}	1.000
Box with obstacle	Legendre	8	3.173×10^2	1.000
Box with obstacle	Chebyshev	8	5.817×10^1	1.000
Box with obstacle	Monomial	10	∞	0.879
Box with obstacle	Legendre	10	9.560×10^2	1.000
Box with obstacle	Chebyshev	10	8.444×10^1	1.000

The complete G_C benchmark contains 48 records: eight degrees, three basis representations, and two constrained regions.

Appendix D

Selected Implementation Excerpts

The complete implementation is submitted separately with the project repository and is available at <https://github.com/k-r-o-y/pal>. These short excerpts document numerical procedures particularly relevant to the experiments.

D.1 Analytical Simplex Moment

Listing D.1: Analytical unit-simplex monomial integral using logarithmic Gamma functions.

```
1 from collections.abc import Sequence
2 from math import exp, lgamma
3 def unit_simplex_monomial_integral(
4     exponents: Sequence[int],
5 ) -> float:
6     if any(alpha < 0 for alpha in exponents):
7         raise ValueError("exponents must be non-negative")
8     dimension = len(exponents)
9     log_numerator = sum(
10         lgamma(alpha + 1.0)
11         for alpha in exponents
12     )
13     log_denominator = lgamma(
14         dimension + sum(exponents) + 1.0
15     )
16     return exp(log_numerator - log_denominator)
```

D.2 Condition Number

Listing D.2: Condition-number estimation from singular values.

```
1 import numpy as np
2 def estimate_condition_number(
3     matrix: np.ndarray,
4 ) -> float:
5     matrix = np.asarray(
6         matrix,
7         dtype=np.float64,
8     )
9     singular_values = np.linalg.svd(
10        matrix,
11        compute_uv=False,
12    )
13    return float(
14        singular_values[0]
15        / singular_values[-1]
16    )
```

D.3 Coefficient Perturbation

Listing D.3: Relative coefficient perturbation used by the controlled experiments.

```
1 import numpy as np
2 def perturb_coefficients(
3     coefficients,
4     delta,
5     rng,
6 ):
7     coefficients = np.asarray(
8         coefficients,
9         dtype=np.float64,
10    )
11    direction = rng.normal(
12        size=coefficients.shape,
13    )
14    direction /= np.linalg.norm(
15        direction
16    )
17    scale = max(
18        np.linalg.norm(coefficients),
19        np.finfo(np.float64).eps,
```

```
20 )
21 return (
22     coefficients
23     + delta * scale * direction
24 )
```

D.4 Basis Count

Listing D.4: Number of total-degree basis functions.

```
1 from math import comb
2 def number_of_basis_functions(
3     dimension: int,
4     degree: int,
5 ) -> int:
6     if dimension < 1:
7         raise ValueError(
8             "dimension must be positive"
9         )
10    if degree < 0:
11        raise ValueError(
12            "degree must be non-negative"
13        )
14    return comb(
15        dimension + degree,
16        degree,
17    )
```

Appendix E

Reproducibility

This appendix records the principal benchmark commands, output artefacts, and environment information needed to regenerate the submitted results. The full implementation and analysis repository is available at <https://github.com/k-r-o-y/pal>.

E.1 Benchmark Commands

Listing E.1: Principal completed benchmark commands.

```
python -m analysis.simplex.run_dimension_benchmark
python -m analysis.simplex.run_precision_benchmark
python -m analysis.downstream.run_downstream_benchmark \
--dimensions 1 2 3 \
--degrees 0 1 2 3 5 8 \
--trials 10 \
--dtypes float32 float64 \
--query-scales 0.25 0.50 0.75 \
--perturbation-magnitude 1e-5
python -m analysis.polytope.run_polytope_benchmark \
--dimensions 2 3 \
--degrees 0 1 2 3 5 \
--trials 10 \
--dtypes float32 float64 \
--schedules shrinking expanding oscillating pulsed random_walk \
--trajectory-steps 12 \
--perturbation-magnitude 1e-5 \
--value-noise-magnitude 1e-5
python -m analysis.pal.run_representative_pal_benchmark
```

E.2 Constraint-Induced Conditioning

Listing E.2: Completed constraint-induced G_C benchmark.

```
python -m analysis.conditioning.run_constraint_gram_benchmark \
--dimensions 2 \
--degrees 0 1 2 3 5 6 8 10 \
--domains unit_simplex box_with_obstacle \
--bases monomial legendre chebyshev
```

The run produces 48 records and writes

```
results/conditioning/constraint_gram_results.csv
results/conditioning/constraint_gram_summary.json
```

The associated plotting command is

Listing E.3: Constraint-induced conditioning plotting command.

```
python -m analysis.plots.plot_constraint_gram_benchmark \
--format pdf png svg \
--plot-rank
```

which produces

```
figures/constraint_gram_condition_number_vs_degree.*
figures/constraint_gram_rank_fraction_vs_degree.*
```

E.3 Plotting and Generated-Polynomial Commands

Listing E.4: Representative plotting commands.

```
python -m analysis.plots.plot_dimension_benchmark \
--degrees 1 2 3 5 \
--format pdf png svg
python -m analysis.plots.plot_precision_benchmark \
--format pdf png svg
python -m analysis.plots.plot_downstream_benchmark \
--format pdf png svg
python -m analysis.plots.plot_polytope_benchmark \
--detail-dimension 3 \
--detail-scenario dynamic_box_with_obstacle \
--detail-schedule oscillating \
--format pdf png svg
python -m analysis.plots.plot_polytope_runtime_vs_dimension \
--input results/polytope/polytope_results.csv \
--scenario dynamic_box_with_obstacle \
--schedule oscillating \
--degree 10 \
--format pdf png svg
python -m analysis.pal.plot_representative_pal_results \
```

```
--results-path results/pal/representative_pal_results.csv \
--summary-path results/pal/representative_pal_summary.json \
--output-dir figures \
--formats pdf png svg
```

Listing E.5: Generated-polynomial visualisation command.

```
python -m analysis.plots.plot_generated_polynomials \
--degrees 1 3 5 \
--trials 0 1 2 \
--format pdf png svg
```

The plotting configuration uses base seed 20260804, density offset 0.1, and source-coefficient scale 0.35. The constrained-polytope plots are regenerated with detail dimension 3, the dynamic box-with-obstacle scenario, and `oscillating` as the named detail schedule. For the polytope degree-dependent figures, the plotting routine aggregates all rows in the selected dimension/scenario across the completed schedules, trajectory steps, and trials before reporting medians and interquartile ranges.

E.4 Result Artefacts

Principal row-level outputs are:

- `results/simplex/dimension_results.csv`;
- `results/simplex/precision_results.csv`;
- `results/downstream/downstream_results.csv`;
- `results/polytope/polytope_results.csv`;
- `results/pal/representative_pal_results.csv`;
- `results/conditioning/constraint_gram_results.csv`.

Corresponding JSON files store benchmark configurations and aggregate summaries. Figures are generated from saved row-level outputs rather than by rerunning the numerical experiments. The regenerated constrained-polytope degree plots use dimension 3 and the dynamic box-with-obstacle scenario; the underlying row-level benchmark itself contains only dimensions 2 and 3 and density degrees 0, 2, 4, 6, 10.

E.5 Repository

The complete implementation, including benchmark runners, basis-conversion and integration utilities, plotting scripts, tests, and result-generation code, is available in the project repository:

<https://github.com/k-r-o-y/pal>

The repository revision recorded below should correspond exactly to the final submitted benchmark state.

E.6 Final Software Environment

For an exact reproduction of the submitted numerical environment, the following commands can be run from the repository checkout used for the benchmark results. They provide the interpreter and package versions, the repository revision, and the local benchmark date without embedding unverified environment-specific values in this document:

Listing E.6: Commands for recording software versions, repository revision, and benchmark date.

```
python --version
python -c "
import numpy
import torch
import pandas
import matplotlib
print('NumPy', numpy.__version__)
print('PyTorch', torch.__version__)
print('Pandas', pandas.__version__)
print('Matplotlib', matplotlib.__version__)
"
git rev-parse HEAD
date "+%Y-%m-%d"
```

E.7 Reproducibility Record

Table E.1: Reproducibility information for the completed benchmark configuration. The table records the verified experimental configuration used by the reported runs. Exact interpreter/package versions, repository revision, and local benchmark date can be captured with Listing E.6 when reproducing the experiments from the corresponding repository checkout.

Component	Value
Operating system	macOS
Hardware	Apple silicon CPU
Project repository	https://github.com/k-r-o-y/pal
Dimension benchmark precision	float64
Dimension benchmark trials	Five per configuration
Dimension benchmark range	$n = 1, \dots, 5$
Precision benchmark arithmetic	float32 and float64
Precision benchmark trials	Ten per configuration
Precision benchmark dimensions	1, 2, 3
Precision perturbation magnitude	10^{-5}
Downstream dimensions	1, 2, 3
Downstream source degrees	0, 1, 2, 3, 5, 8
Downstream density degrees	0, 2, 4, 6, 10, 16
Downstream query scales	0.25, 0.50, 0.75
Downstream trials	Ten per configuration
Downstream density offset	0.1
Downstream coefficient scale	$0.35/(1 + \alpha)$
Polytope dimensions	2, 3
Polytope source degrees	0, 1, 2, 3, 5
Polytope density degrees	0, 2, 4, 6, 10
Polytope records	37200
Polytope schedules	Shrinking, expanding, oscillating, pulsed, and random walk
Polytope trajectory steps	12 per dynamic schedule
Polytope detail-plot dimension	3
Polytope detail-plot scenario	Dynamic box with obstacle
Polytope named detail schedule	Oscillating

Continued on the next page.

Table E.1 continued from the previous page.

Component	Value
Polytope degree-plot aggregation	Median and IQR across all dynamic schedules, trajectory steps, and trials within the selected dimension/scenario
Polytope recovery oversampling factor	2.0
Polytope value-noise magnitude	10^{-5}
Representative PAL dimensions	2, 3
Representative PAL source degrees	0, 1, 2, 3, 5
Representative PAL density degrees	0, 2, 4, 6, 10
Representative PAL trials	Ten per configuration
Representative PAL records	600 finite records
Constraint-induced G_C dimension	2
Constraint-induced G_C degrees	0, 1, 2, 3, 5, 6, 8, 10
Constraint-induced G_C domains	Unit simplex and box with obstacle
Constraint-induced G_C records	48
Precision-sensitive PAL triangle	One degree-12 monomial triangle reproducer
Triangle default-path result	$-1.7947995740423999 \times 10^{-24}$
Triangle full-float64 result	$+7.2752680397899133 \times 10^{-24}$